
Livello Applicazione

Esempio: World Wide Web

Internet, Web e Cloud
Claudia Canali

Il livello applicazione utilizza il sottostante livello di trasporto per far comunicare processi in esecuzione su host terminali e realizzare **applicazioni (o servizi) di rete**

Esempi di applicazioni di rete

- E-mail
- Web
- P2P file sharing
- Text messaging
- Multi-user network games
- Streaming video / audio (es. YouTube, Hulu, Netflix)
- Real-time videoconferencing video / audio (es. Skype)
- Social Networking
- Motori di ricerca

**Da non confondere
con protocolli di rete**

Creare una applicazione di rete



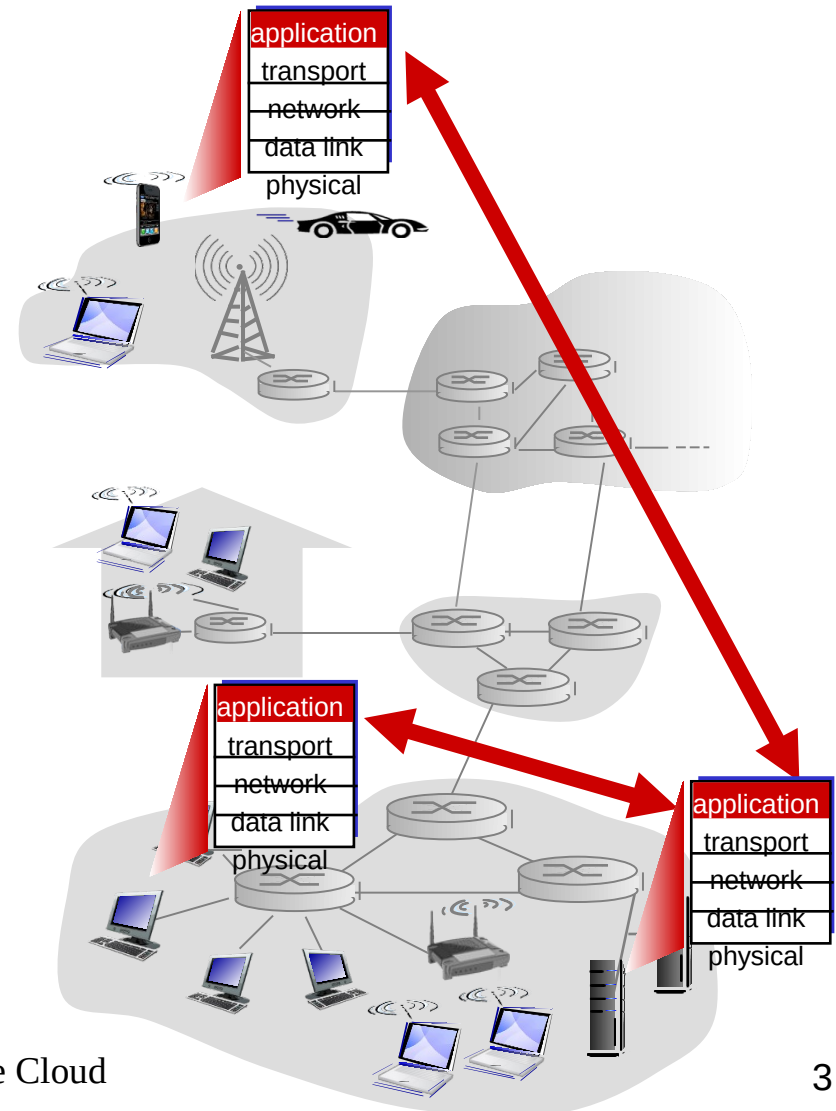
Scrivere un programma che:

- Può girare su sistemi eterogenei (host)
- Comunica attraverso la rete
- Es. Web browser che comunica con il software di un Web server

Nessun bisogno di scrivere software per i dispositivi del network core

- Sui router nel network core non girano applicazioni utente (di livello applicazione)

→ ***Rapido sviluppo di una vasta gamma di applicazioni di rete!***



Architettura di una applicazione di rete



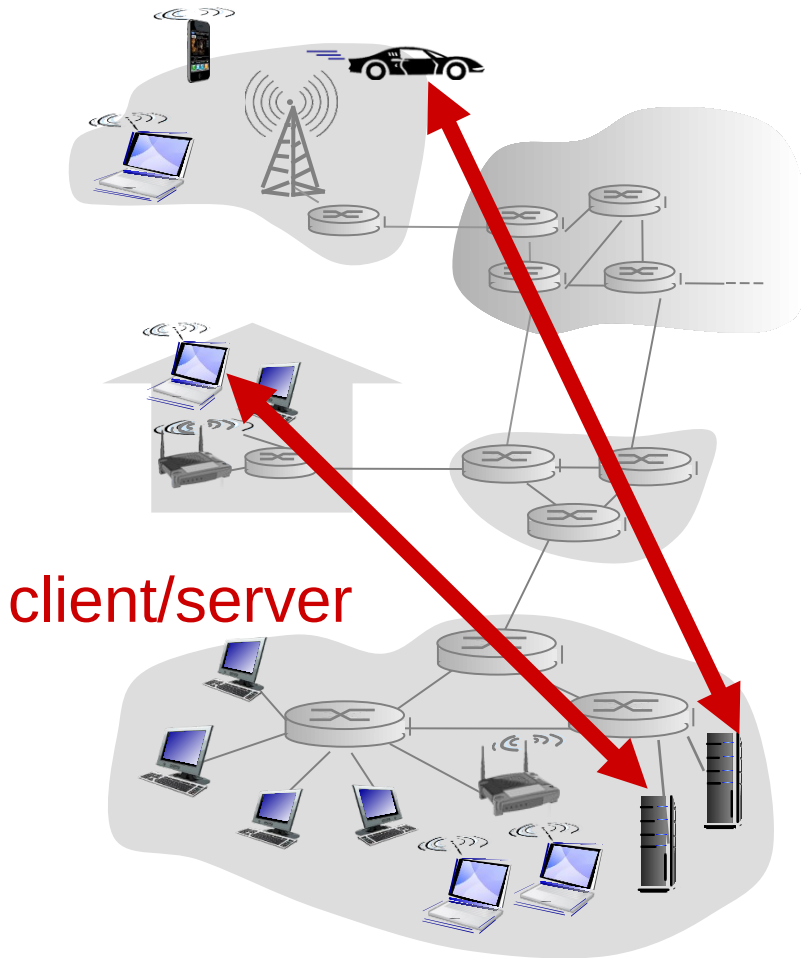
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Due principali alternative attualmente utilizzate:

1) Client-server

2) Peer-to-peer (P2P)

Architettura client-server



Client:

- Comunica col server
- **Non comunicano tra di loro direttamente**
- Possono essere connessi in maniera **intermittente**
- Possono avere **indirizzi IP dinamici**

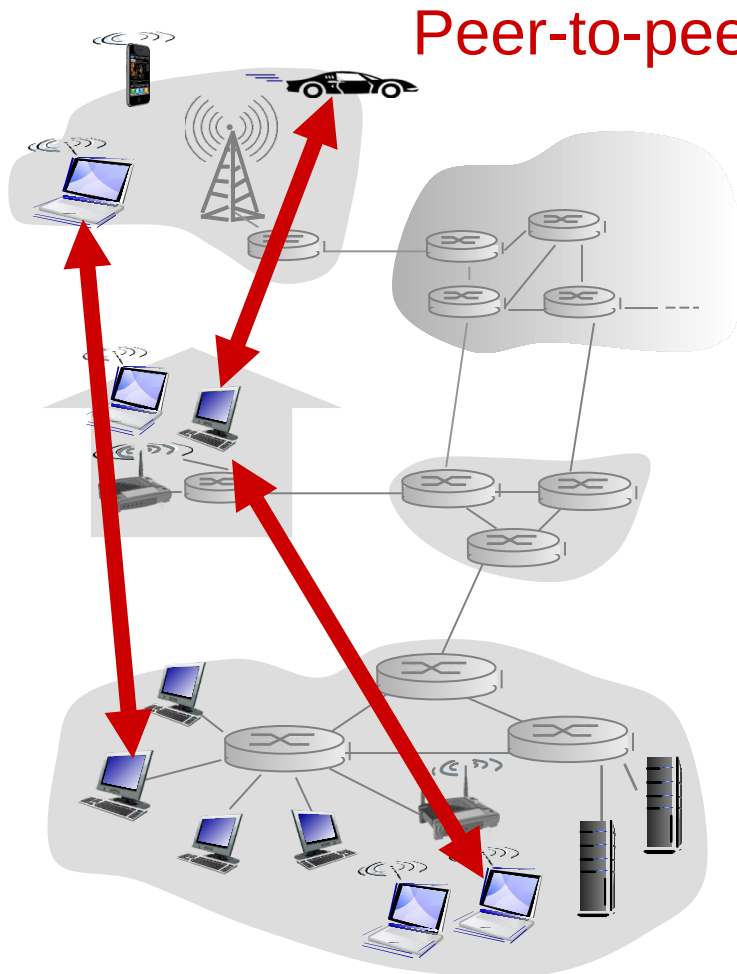
Server:

- Sempre **attivo e in ascolto** di richieste (es. Web server)
- **Indirizzo IP fisso e noto**
- **Data center per scalabilità** (es. Google centinaia di data center – ognuno con centinaia di migliaia di server – per supporto ai vari servizi Gmail, Drive, Youtube, ...)

Esempi:

- Web, trasferimento file FTP, posta elettronica, ...

Architettura P2P



Peer-to-peer

- **Comunicazione diretta** tra coppie di host (peer)
- I peer sono **connessi a intermittenza e cambiano indirizzo IP**
 - Proprietà di utenti – non di fornitori di servizi
- I peer fungono **sia da client che da server** (chiedono servizi ad altri peer e allo stesso tempo ne offrono)
- Concetto di **self scalability**: nuovi peer portano nuova capacità di servizio così come nuova richiesta di servizi
- **Economicamente** meno dispendiose
- **Grandi sfide** legate a sicurezza, prestazioni e affidabilità
- **Esempio più famoso**: file sharing (BitTorrent – nel 2004 occupava il 35% del traffico Internet)
- **Architetture ibride** con login server e peer (e super-peer): telefonia e videoconferenza su Internet (Skype in anni passati, ora cloud)

Processo: programma in esecuzione su un host

- **Sullo stesso host**, 2 processi comunicano usando un approccio interprocesso (**inter-process communication**) definito dal SO
- **Su host diversi** 2 processi comunicano scambiando **messaggi** attraverso la rete ← *nostro focus*

Applicazioni di rete: coppia di processi che scambiano messaggi su una rete

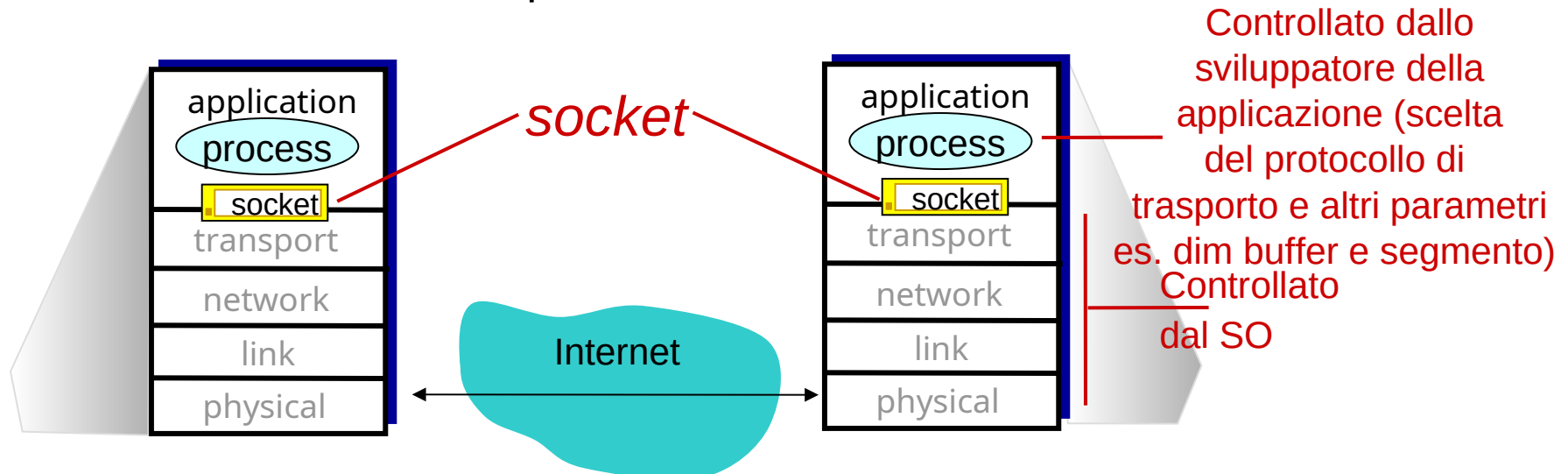
Client / Server

- **Processo client:** processo che **inizia** la comunicazione
- **Processo server:** processo **in attesa di essere contattato** che riceve la comunicazione e (se previsto) risponde

P2P: applicazioni con architettura P2P hanno sia processi client che server (**utilizzabili termini lato client e lato server**)

Socket / porte

- Il processo invia/riceve messaggi alla/dalla rete attraverso un'interfaccia software detta **socket** (analoga a una **porta**)
- **Socket** = interfaccia tra processo a livello applicazione e livello trasporto in un host (**API** per costruire applicazioni di rete)
 - Il processo mittente invia i messaggi in uscita sulla socket
 - Il processo mittente si appoggia sull'infrastruttura di trasporto sull'altro lato della porta per consegnare i messaggi fino alla socket del processo destinatario



Per ricevere messaggi, il processo destinatario deve avere un **identificatore**

- **Indirizzo IP**: numero di 32 bit (4 bytes → 4 numeri interi separati da un punto) che identifica univocamente un host
 - Sufficiente per identificare uno specifico processo in esecuzione sull'host corrispondente? *No, più processi possono girare sullo stesso host*
- **Identificatore**: deve includere **sia indirizzo IP che numero di porta** associata al processo sull'host
- Alle applicazioni più note sono assegnate porte specifiche.
- Esempi:
 - HTTP server: 80
 - Mail server (SMTP): 25
- Numeri di porte associati a protocolli internet: www.iana.org
- Mandare messaggio al server Web su gaia.cs.umass.edu: IP 128.119.245.12 porta 80

Quali requisiti hanno le applicazioni di rete?



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

*Quali requisiti possono avere le applicazioni di rete?
(ossia, come scegliere il servizio di trasporto per un'applicazione?)*

4 requisiti principali

1) Trasferimento dati affidabile (integrità dei dati consegnati corretti ei completi)

- Alcune applicazioni necessitano **trasferimento al 100% affidabile**. Es. trasferimento file, transazioni Web, posta elettronica)
- Altre applicazioni possono **tollerare alcune perdite di dato**. Es. applicazioni audio/video streaming tollerano piccoli difetti in multimedia riprodotti

Cause possibili di perdite di pacchetti

- Buffer di output di un router pieno → **Buffer overflow**
 - Il pacchetto non può essere memorizzato, quindi è scartato e perso
- Errori di trasmissione a livello di bit → scartato da un router o un host

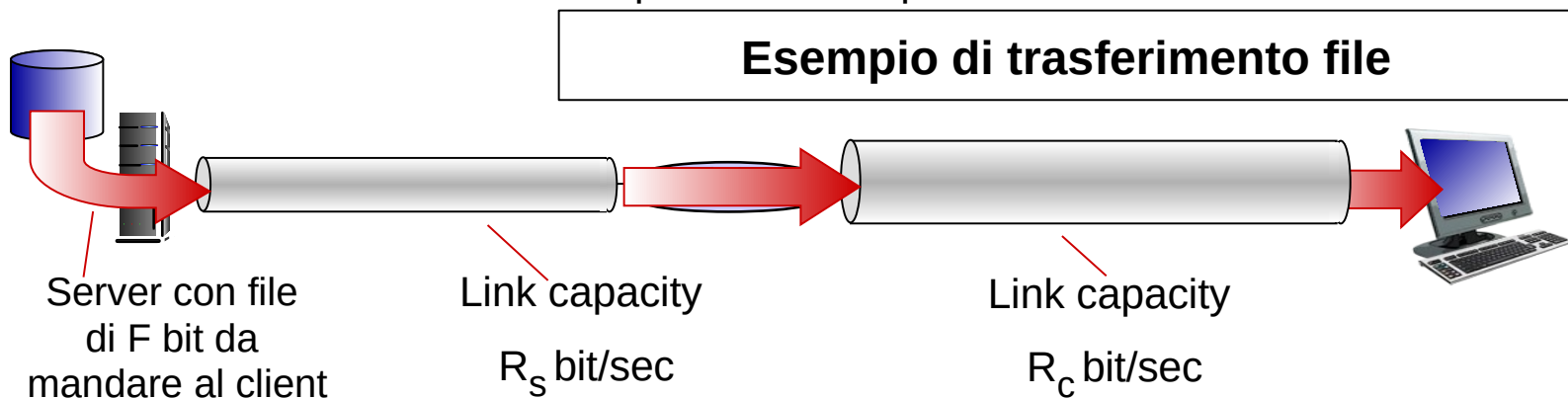
Throughput

2) Throughput end-to-end

- Alcune applicazioni (es. voice over IP) per essere 'efficaci' richiedono un **throughput** (tasso di invio di bit al processo destinatario) **minimo garantito**
 - Applicazioni **bandwidth-sensible**
- **Applicazioni elastiche:** non hanno specifici requisiti di banda

Throughput: rate (bit/secondo) a cui i bit sono trasferiti tra mittente e destinatario
– *misura critica prestazioni di rete*

- **istantaneo:** rate ad ogni istante di tempo
- **medio:** rate medio misurato su un periodo di tempo



Throughput



- Se $R_c > R_s$: throughput end-to-end $\rightarrow R_s$
- Se $R_s > R_c$: throughput end-to-end $\rightarrow R_c$
- Il link di **capacità minore $\min(R_c, R_s)$** definisce il throughput end-to-end
 $\rightarrow$ collo di bottiglia (**bottleneck link**) del collegamento

**Rischio di
buffer overflow**

Esempio specifico

- File MP3 di $F=32$ milioni di bit
- $R_s = 2$ Mbps velocità di trasmissione del server
- $R_c = 1$ Mbps velocità di accesso del client (***throughput end-to-end***)
- **$T = 32$ sec tempo necessario per il trasferimento**

Osservazione: spesso il bottleneck è la rete di accesso

Temporizzazione e Sicurezza



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

3) Temporizzazione

- Alcune applicazioni **time sensitive** richiedono una **bassa latenza per essere 'efficienti'** (es. telefonia Internet, teleconferenze, videogiochi interattivi)
- Garanzie che ogni bit sia ricevuto dal destinatario con latenze basse e a bassa variabilità dopo la trasmissione

4) Sicurezza

- Un protocollo di trasporto può offrire uno o più servizi di sicurezza
- Es. Cifratura lato mittente, integrità dati e autenticazione

Requisiti delle applicazioni di rete



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

<u>Applicazione</u>	<u>Perdita dati</u>	<u>Throughput</u>	<u>Time sensitive</u>
File transfer	no loss	elastic	no
E-mail	no loss	elastic	no
Web documents	no loss	elastic	no
Real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	sì, 100 msec
Stored audio/video	loss-tolerant	Come sopra	sì, pochi sec
Interactive games	loss-tolerant	Pochi Kbps	sì, 100 msec
Text messaging	no loss	elastic	sì e no

Protocolli di trasporto



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

TCP

Trasporto affidabile tra processi mittente e destinatario

Controllo di flusso: mittente non sovraccarica il destinatario

Controllo di congestione: varia velocità di trasmissione se la rete è congestionata

Non offre: temporizzazione, minimum throughput garantito, sicurezza

Connection-oriented: setup richiesto tra processi client e server

UDP

Trasporto non affidabile tra processi mittente e destinatario

Non offre: flusso di controllo, controllo di congestione, timing, *temporizzazione*, *minimum throughput garantito*, sicurezza, o setup di connessione

Nota: temporizzazione, minimum throughput non garantiti ma migliori del TCP

Protocolli usati



Applicazione	Protocollo di livello applicazione	Protocollo di trasporto
File transfer	FTP	TCP
E-mail	SMTP	TCP
Web	HTTP	TCP
Streaming multimedia	RTP HTTP (es. YouTube)	UDP o TCP
Telefonia mobile		
Videoconferenza	SIP, RTP, proprietario	UDP

World Wide Web: nascita ed evoluzione

Web: motivi della nascita



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Prima degli anni '90: Internet era relegata a essere una rete dedicata alle comunicazioni all'interno della comunità scientifica e tra le associazioni governative e amministrative

Tim Berners-Lee (CERN di Ginevra)

- “The current **incompatibilities of the platforms and tools** make it impossible to access existing information through a common interface, leading to **waste of time**, ...”
- “A link is specified as an **ASCII string** from which the browser can deduce a suitable method of contacting an appropriate server. When a link is followed, the browser addresses the request for the node [document] to the server.”



*Nasce il **World Wide Web...***

Il Web in origine



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Sistema che permette una **lettura ipertestuale, non-sequenziale** dei documenti, saltando da un punto all'altro mediante l'utilizzo di rimandi (link o, più propriamente, **hyperlink**)

Il 6 agosto Berners-Lee pubblicò il **primo sito Web della storia**, presso il CERN
<https://web.archive.org/web/20150717103715/http://info.cern.ch/hypertext/WWW/TheProject.html>

Nel **World Wide Web (WWW)**, le risorse disponibili sono organizzate secondo un **sistema di librerie, o pagine**, a cui si può accedere utilizzando appositi programmi detti **Web browser** con cui è possibile **navigare** visualizzando file, testi, ipertesti, suoni, immagini, animazioni e filmati

**Inizialmente:
poche centinaia
di pagine HTML**

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents. Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#) , [Policy](#) [What's out there?](#)
Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.
[Help](#)
on the browser you are using
[Software Products](#)
A list of W3 project components and their current state. (e.g. [Line Mode](#) ,X11 [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail robot](#) , [Library](#))
[Technical](#)
Details of protocols, formats, program internals etc
[Bibliography](#)
Paper documentation on W3 and references.
[People](#)
A list of some people involved in the project.
[History](#)
A summary of the history of the project.
[How can I help ?](#)
If you would like to support the web..
[Getting code](#)
Getting the code by [anonymous FTP](#) , etc.

I veri motivi alla base del successo iniziale

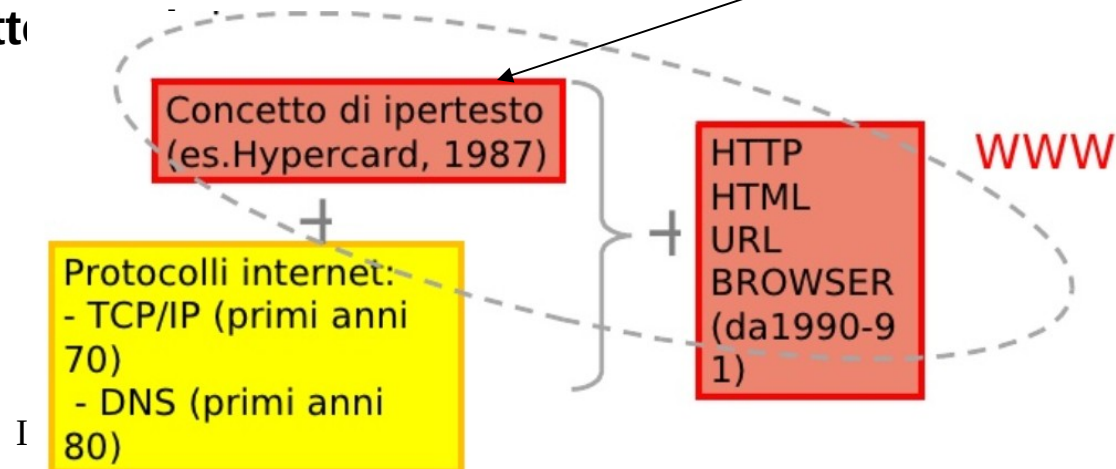


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Digitalizzazione dell'informazione**
 - (Qualsiasi informazione come sequenza di 0 e 1)
- **Diffusione di Internet (dagli anni '70)**
 - (Trasporto dell'informazione ovunque, in tempi rapidissimi e a costi bassissimi)
- **Diffusione dei PC (dagli anni '80)**
 - (Accesso, memorizzazione ed elaborazione dell'informazione da parte di chiunque a costi bassissimi)
- **Interfacce utente semplificate (dagli anni '80)**
- **Presenza delle tecnologie sott**

HyperCard: software ipertestuale introdotto da Apple

"I just had to take the hypertext idea and connect it to the TCP and DNS ideas and—ta-da!—the World Wide Web" Berners-Lee



Il Web 1.0 - statico



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

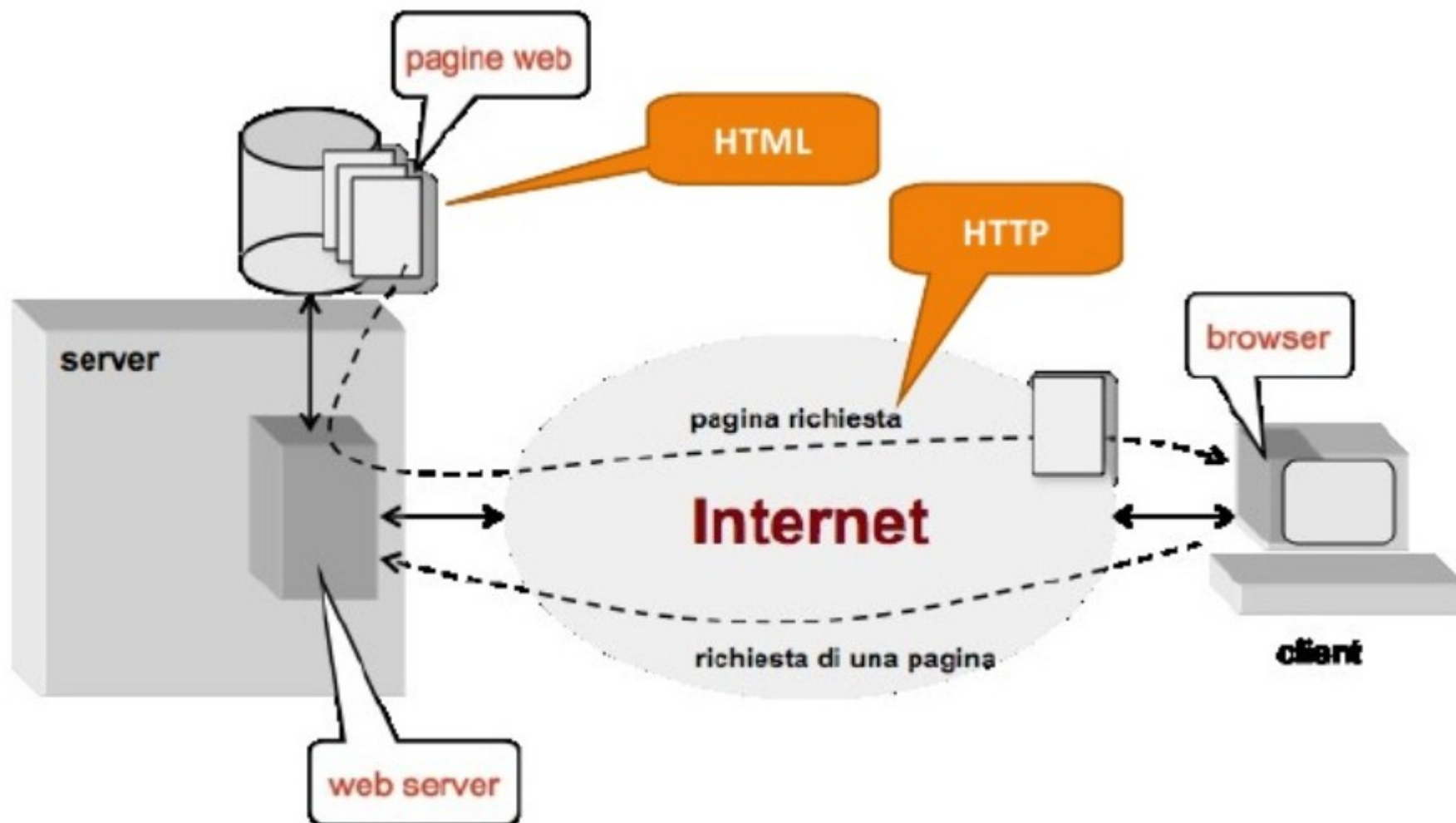
All'inizio il Web era **statico** - l'utente **non interagiva con il contenuto**: gli URL corrispondevano a documenti HTML che venivano restituiti al client

- ❑ Siti e portali con contenuti prodotti dal publisher
- ❑ Organizzazione gerarchica dell'informazione e navigazione attraverso menu
- ❑ Data base
- ❑ Interazione sito ↔ singolo utente
- ❑ Directory e motori di ricerca
- ❑ E-commerce
- ❑ Servizi "chiusi" ("attrarre l'utente sul sito, e tenercelo")
- ❑ Banda stretta

Web publishing

Prima forma di interattività
Web dinamico

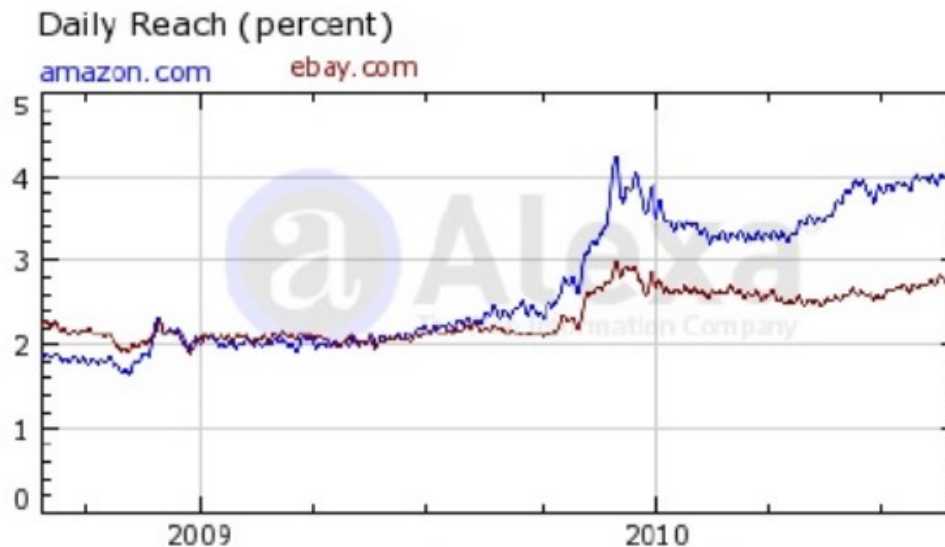
Architettura di base



E-commerce: gli albori

★ <http://www.amazon.com>
(dal 1995, <http://en.wikipedia.org/wiki/Amazon.com>)

□ <http://www.ebay.com>
(dal 1995, <http://en.wikipedia.org/wiki/EBay>)



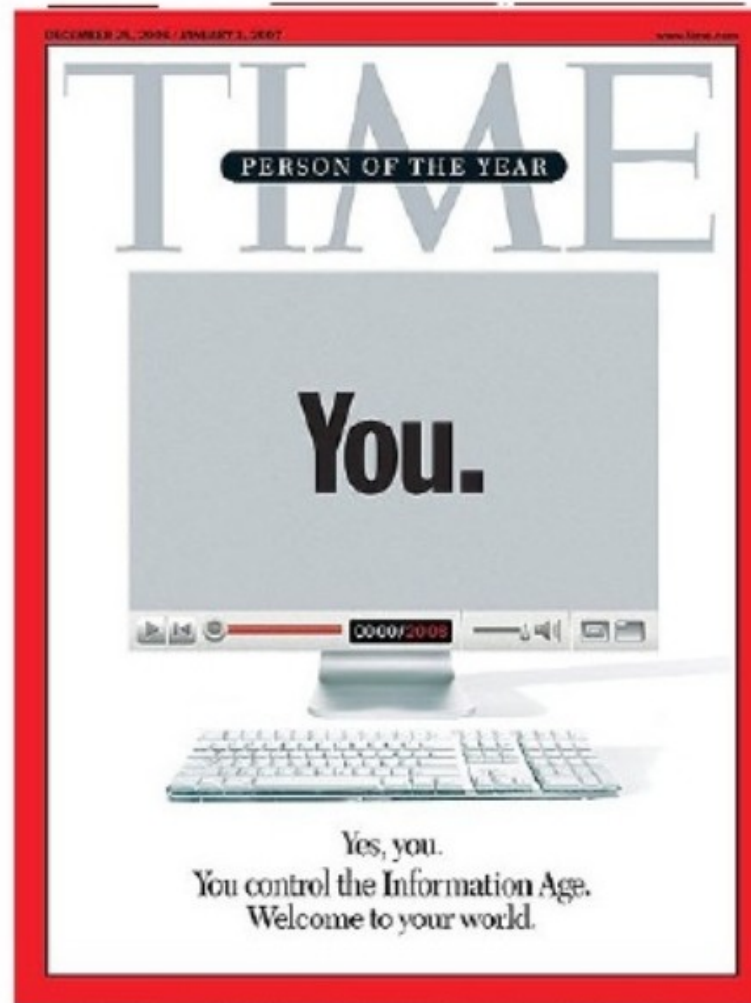
Revenues 2009:
- **Amazon 24,5 B\$**
(+28%)
- **eBay 8,7 B\$**
(+2%)

- Termine coniato nel **2004** – **prima Web 2.0 Conference**
- **“Web as a platform”**: i siti Web 2.0 sono piattaforme che consentono grande interazione tra utenti
- **User-centric**: gli utenti forniscono il loro valore aggiunto con l'autoproduzione di contenuti e la condivisione della conoscenza
 - Non più utilizzatori passivi ma creatori di contenuto
 - Centralità e protagonismo dell'utente
 - Comunicazione 'da uno a molti' → 'da molti a molti'
- Da Web 'read only' a Web 'read-write'
- Anche detto ***Social Web***

Time, 2006



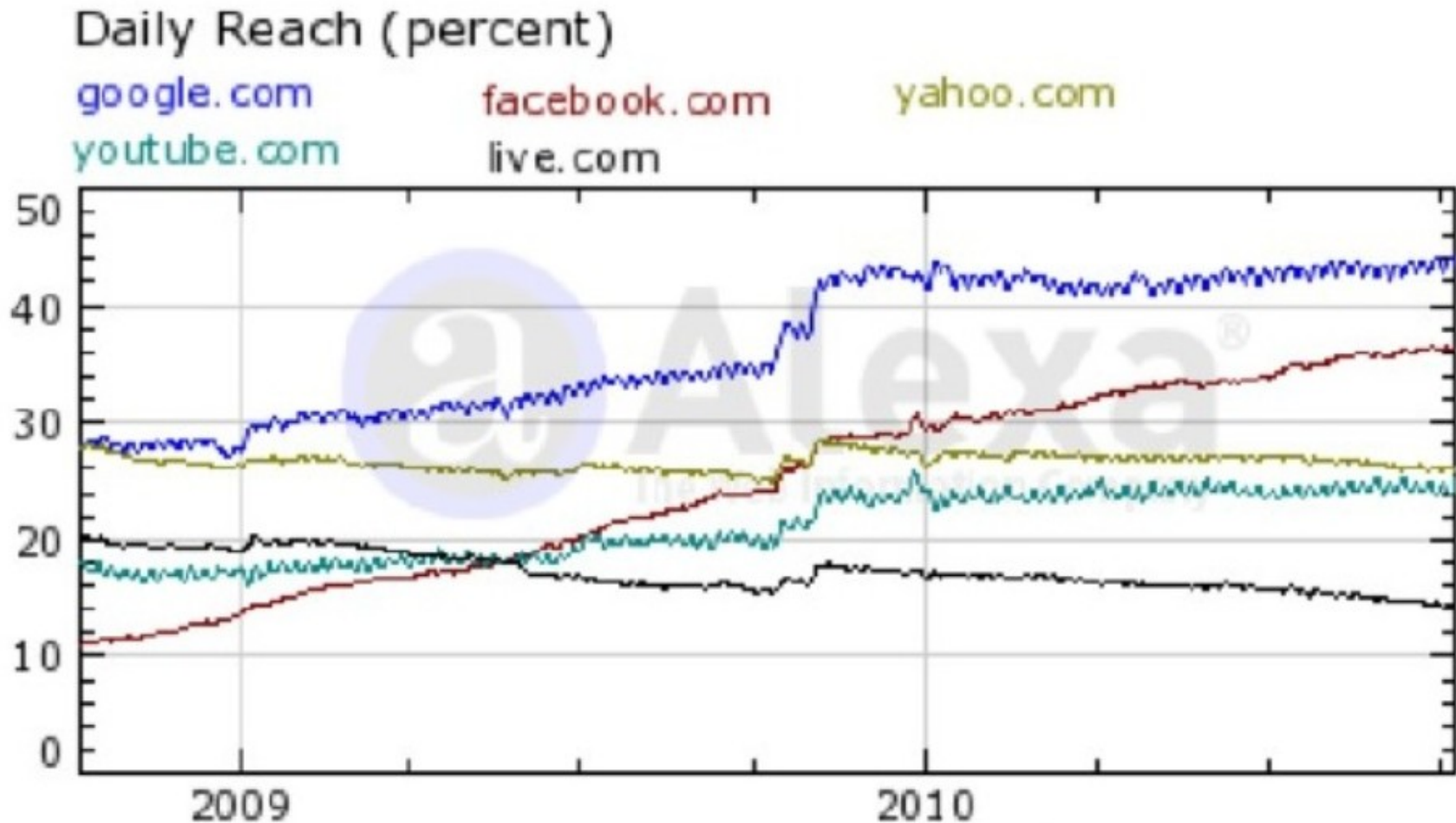
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Top 5 sites (2010)



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



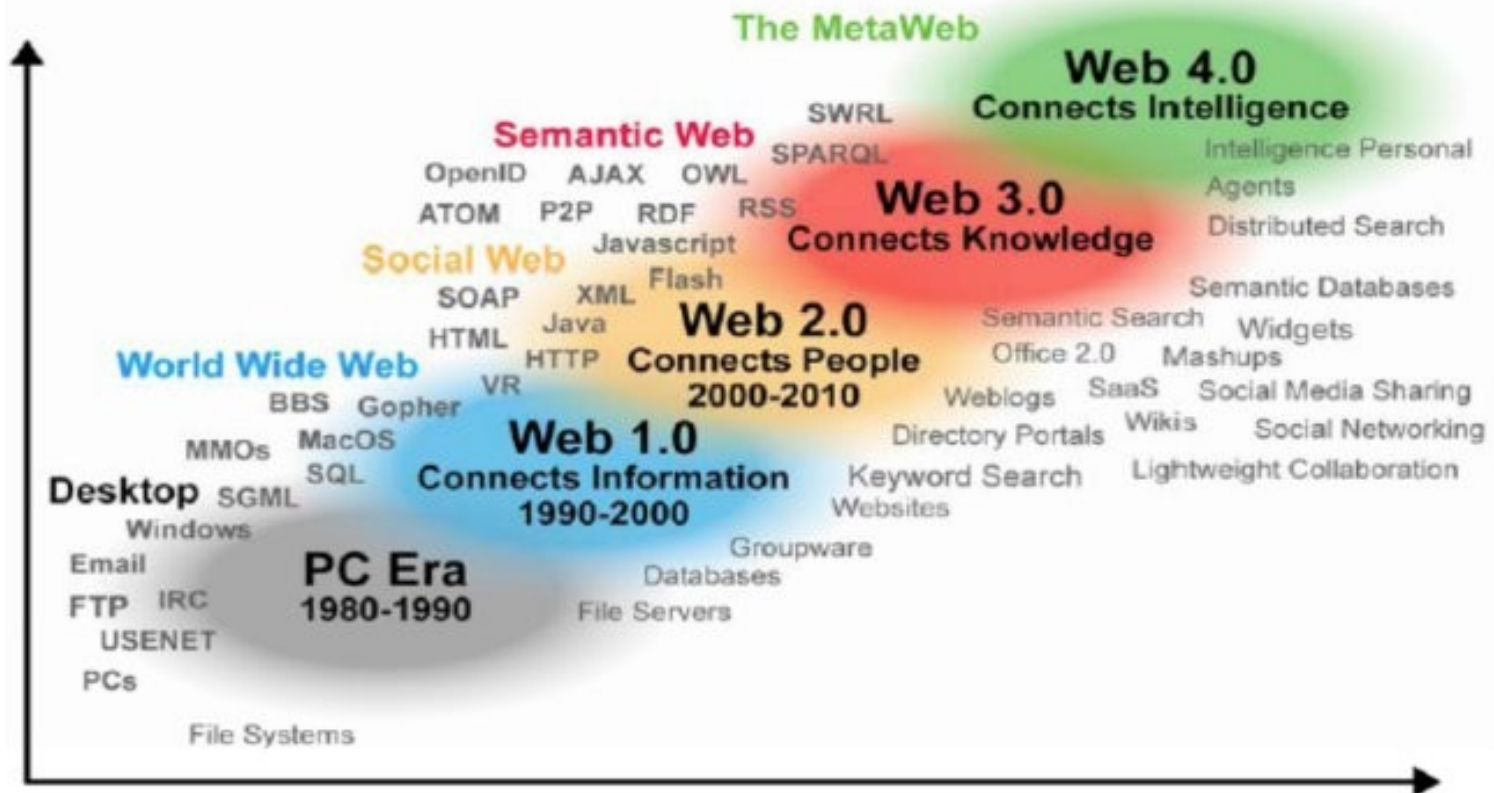
- **Web 3.0 → Semantic Web**
- Algoritmi intelligenti sono incorporati nei sistemi Web per interpretare dati e fornire **servizi in una forma più strutturata e personalizzata per ciascun utente**
- Analizzano i dati storici degli utenti e li utilizzano per migliorare la successiva esperienza di navigazione sul Web
 - Es. Sistemi di recommendation: consigli su altri articoli da acquistare in base alle precedenti ricerche e/o acquisti dell'utente stesso e di altri utenti caratterizzati come "simili"
 - Es. Servizi personalizzati sulla posizione geografica dell'utente (meteo, promozioni, eventi, ...)
- Apprendimento automatico, reti neurali e intelligenza artificiale, Internet of Things (IoT) ecc., sono alcune delle tecnologie caratteristiche del Web 3.0

Web X.0



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Le definizioni si fanno un pò più confuse
- *Ma le tecnologie sottostanti?*



Il Web dinamico



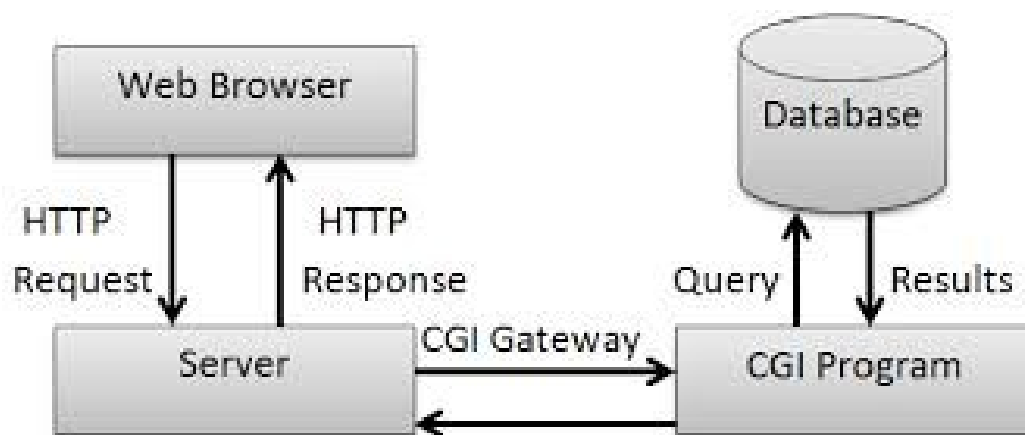
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Web dinamico: applicazioni Web che restituiscono pagine dinamiche, ossia che contengono informazioni modificate (personalizzate) in base a dati ricevuti dal client/utente

Primo strumento di **Web dinamico**: uso di **Common Gateway Interface (CGI)** - standard per l'interfacciamento dei server Web con applicazioni esterne per la generazione di contenuti dinamici

URL corrispondenti a programmi CGI eseguiti dal server in tempo reale: il programma CGI interagisce con il DB per generare il contenuto dinamico – personalizzato

Il programma CGI restituisce al server la pagina HTML che contiene informazioni personalizzate, che il server restituisce al client



CGI

- Programma che può essere scritto con **qualsiasi linguaggio di programmazione** (più usati: C e Perl)
- **Standard di comunicazione** col server circa la modalità per scambiare parametri di input (variabili di ambiente, riga di comando o std input) e l'output

Problema: scalabilità!

- Un nuovo processo invocato per ogni richiesta al server
→ **carico pesante** sul server al crescere della quantità di richieste
- Negli anni 1995-1996 la crescita di pagine Web dinamiche aumenta in modo significativo con l'avvento del commercio elettronico
- *E ci sono altri problemi con CGI...*

Esempio CGI



Consideriamo uno script CGI scritto in Python...

```
#!/usr/bin/env python  
  
import MySQLdb  
  
print("Content-Type: text/html\n\n")  
  
print("<html><head><title>Books</title></head><body><ul>")  
  
connection = MySQLdb.connect(user='me', passwd='letmein', db='my_db')  
  
cursor = connection.cursor()  
  
cursor.execute("SELECT name FROM books ORDER BY pub_date DESC  
    LIMIT 10")  
  
for row in cursor.fetchall():  
    print("<li>%s</li>" % row[0])  
  
print("</ul></body></html>")  
  
connection.close()
```

Cosa può
rappresentare un
potenziale problema?

Esempio CGI



Consideriamo uno script CGI scritto in Python...

```
#!/usr/bin/env python
import MySQLdb
print("Content-Type: text/html\n\n")
print("<html><head><title>Books</title></head><body><ul>")
connection = MySQLdb.connect(user='me', passwd='letmein', db='my_db')
cursor = connection.cursor()
cursor.execute("SELECT name FROM books ORDER BY pub_date DESC
LIMIT 10")
for row in cursor.fetchall():
    print("<li>%s</li>" % row[0])
print("</ul></body></html>")
connection.close()
```

Riferimento
esplicito a DB con
credenziali incluse

SQL inserito nello
script

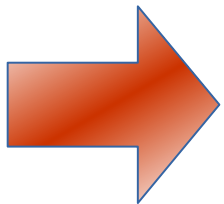
HTML mescolato
a logica applicativa

Problemi CGI



Consideriamo l'esempio precedente analizzandolo in un'ottica di progetto complesso

- Codice potenzialmente ripetuto: connessione e interrogazioni a DB in SQL
- Codice di dettaglio: settare content-type
- Codice poco riusabile (credenziali DB hardcoded)
- Combinazione codice Python e HTML
- Combinazione di output e application logic



Sconsigliabile anche per un **progetto piccolo one-of-a-kind!**

Tecnologie di scripting

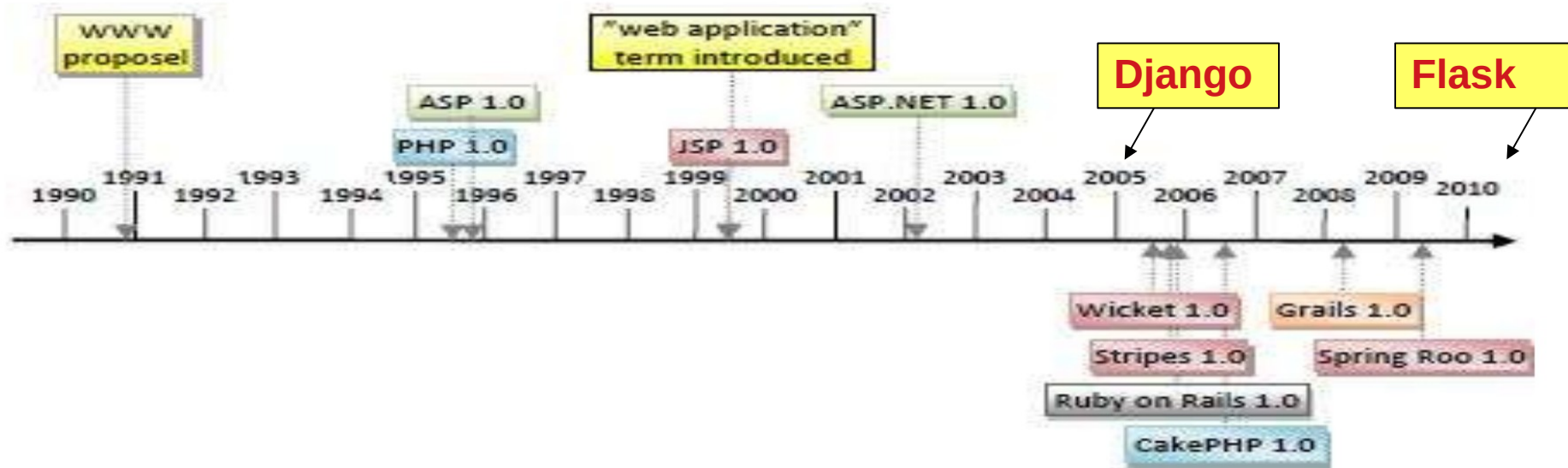


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- CGI → scalabilità limitata
- Limite alla scalabilità: dovuto alla necessità di attivazione di un nuovo processo per ogni richiesta dinamica
- Per aumentare le prestazioni, servono tecnologie che evitino la creazione di un nuovo processo ad ogni richiesta
- Nascono le **tecnologie di scripting**
 - **FastCGI**: permette di condividere un'istanza di un programma CGI
 - **Server API** (Netscape NSAPI, MS ISAPI): Librerie condivise caricate nello spazio del server HTTP, in grado di servire richieste multiple senza creare nuovi processi. Ma... **poca portabilità** (legate al server HTTP) e **vulnerabili**
 - **mod_perl, mod_python, mod_php**: moduli di Apache, in grado di interpretare script all'interno del processo HTTP

- PHP: Hypertext Preprocessor
- Nato a metà anni '90: in origine una raccolta di script CGI
- Evoluzione: possibilità di integrare il codice PHP nel codice HTML per semplificare la creazione di pagine dinamiche
 - Usato per funzionalità di accesso al Database e processing dati
- Grande popolarità
 - Nel **1997** circa 50mila siti Web erano basati su PHP
 - **1998** PHP 3.0 installato sul 10% dei server Web in Internet
 - 2001 supera il milione di siti che lo utilizzano
 - Attualmente ancora molto popolare per applicazioni Web lato server
 - *Gran quantità di siti Web ancora scritti in PHP*
 - *CMS popolari (Wordpress, Drupal, ...) basati su PHP*

Evoluzione....

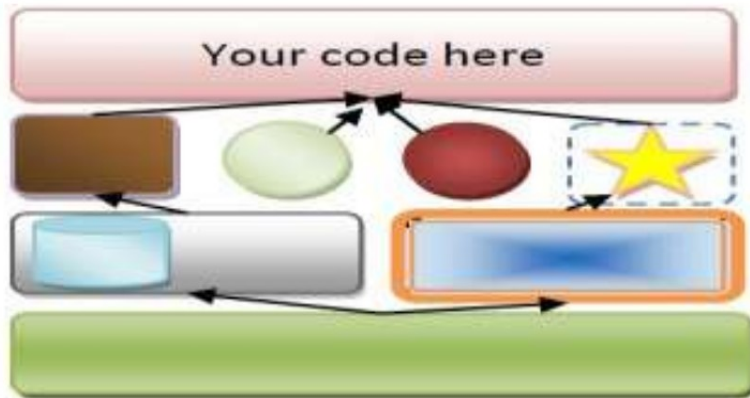


... verso i Framework Web

- Dal 2002 in poi: nascita e sviluppo di Framework Web
- Django nasce nel 2005

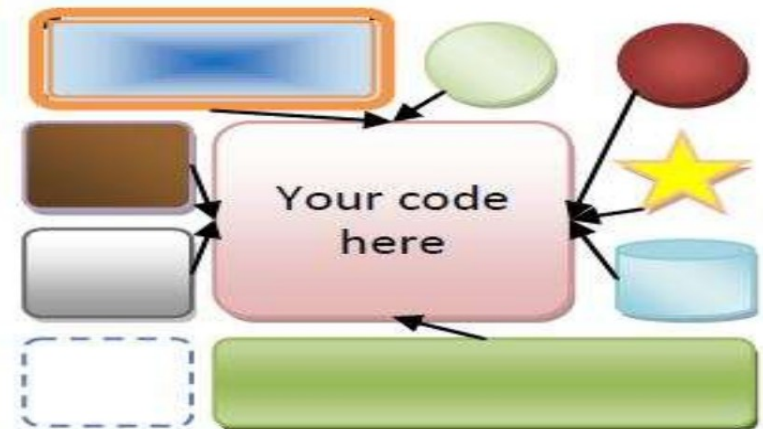
Framework Web

- **Framework Web** progettati per supportare lo sviluppo di siti Web dinamici
- Include librerie che offrono funzionalità comuni a questo tipo di applicazioni
- Base o scheletro su cui costruire



framework

VS



Library

Vantaggi dei Framework Web



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Tutte le applicazioni Web moderne hanno un **set comune di requisiti**: user management (es., secure user login, password recovery), group management, funzioni di autenticazione/autorizzazione, funzioni admin, supporto per la sicurezza (protezione da attacchi più comuni)
- Un **Web Application Framework** supporta già queste funzionalità, lasciando la possibilità agli sviluppatori di focalizzarsi sugli specifici requisiti delle loro applicazioni
- **Web Application Framework** offrono supporto per interazione trasparente con i più popolari database relazionali (*e non solo*) sia in termini di creazione della base di dati che di funzioni per l'accesso e la manipolazione dei dati contenuti nel DB

Toccheremo con mano i vantaggi con Flask/Django

Gli ingredienti del Web

Web: Ingredienti



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Sistema client-server**
- **“Solo” tre nuovi standard**
 - **URL**: Sistema di indirizzamento delle risorse
 - **HTML**: Linguaggio di markup ipertestuale
 - **HTTP**: *Protocollo di livello applicazione*
- **Meccanismi di comunicazione e namin** **di Internet**
 - Protocollo **TCP/IP** ←
 - Sistema **DNS**

**Standard per
formato documenti**

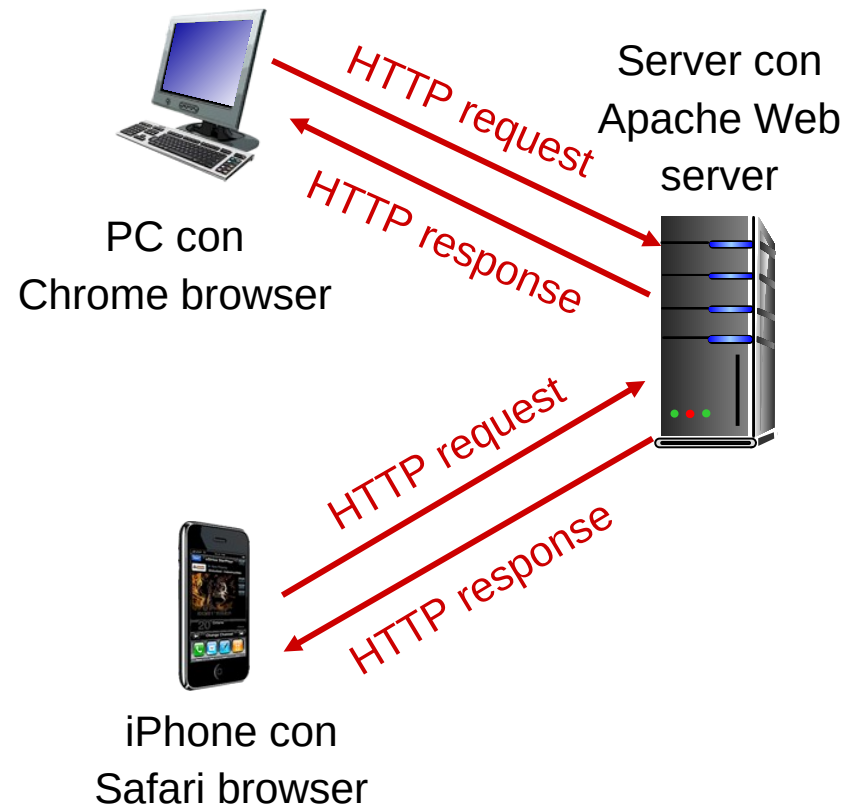
Già presenti

Un protocollo di livello applicazione (in generale) definisce:

- Tipi di messaggi scambiati (richiesta, risposta)
- Sintassi dei messaggi (campi del messaggio)
- Semantica dei campi
- Regole comunicazione (quando e come i processi inviano richieste e rispondono)

Client-server nel Web

- **HTTP: HyperText Trasfer Protocol**
- Protocollo di livello applicativo che permette il reperimento delle risorse Web
- **Modello client/server**
 - **Client**: browser che invia richieste e riceve le risposte (attraverso il protocollo HTTP) e visualizza gli oggetti Web ricevuti
 - **Server**: Web server che invia oggetti Web (attraverso il protocollo HTTP) in risposta a richieste ricevute



URL e pagine Web



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Il meccanismo standard di naming delle risorse Web è l'**Uniform Resource Locator (URL)**
 - Specifica la **locazione fisica** di ogni risorsa presente nel Web
- Una pagina Web è costituita da **diversi oggetti**:
 - Pagine HTML (solitamente il **file principale**)
 - Immagini
 - Audio
 - Video
 - Programmi eseguibili
 - ...

**Il file HTML referencia
gli altri oggetti nella pagina
tramite il loro URL**

Ogni oggetto è identificato dal proprio URL

Campi dell'URL

schema :// host.domain / pathname

http :// www.dsi.unimo.it / docenti/esami.html

Equivalente

- **Schema**: indica il modo con cui accedere alla risorsa, cioè quale protocollo bisogna usare per interagire con il server che controlla la risorsa
 - Metodo di accesso più comune: **HTTP** (protocollo nativo del WWW per il recupero di risorse Web)
- **host.domain**: hostname del nodo (server) su cui risiede la risorsa Web
- **pathname**: identifica la risorsa presso il server Web (percorso della risorsa)

http:// www.dsi.unimo.it:80 / docenti/esami.html

Alcune definizioni



- **Richiesta di risorsa / pagina Web:** una singola richiesta effettuata dall'utente per un determinato URL
 - **Sessione utente:** serie di richieste di risorse effettuate dallo stesso utente al medesimo sito Web durante un intervallo di tempo
 - **Natura composta delle pagine Web**
 - Ogni pagina è composta da diversi oggetti 'embedded'
 - Elementi multimediali o interattivi incorporati all'interno della pagina
 - Possono provenire da fonti esterne o essere 'locali' al sito stesso
 - Esempi:
 - File multimediali (audio, video, animazioni)
 - Documenti e file di lavoro (es., pdf, google doc, sheet, ...)
 - Iframe (per incorporare altre – parti di – pagine Web)
 - Script (es. Javascript, ...)
- *tipicamente una richiesta di pagina Web consiste di multiple richieste per singoli oggetti embedded inviate dal client dell'utente al server Web che ospita il sito*

Tipica Pagina Web



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Pagina HTML + *embedded objects* in file separati

Video

CORONAVIRUS
Domenica 15 alle 21 tutti alla finestra con una torcia

Testaccio

ITALIA
Coronavirus: ecco tutte le regole nei luoghi di lavoro

FINANZA
Doris (B.Mediolanum): nervi saldi e investire a rate

ITALIA
Coronavirus: flashmob, applauso a medici

CORONAVIRUS & FILANTROPIA
Finanza e fintech a caccia di fondi per la solidarietà - Il mondo dell'energia si mobilita
di Lucilla Incorvati e Federica Pezzatti

LUTTI
Scomparso Vittorio Gregotti, il Dominus dell'architettura italiana del XX secolo - Il ricovero per polmonite
di Fulvio Irace

SANITÀ
Quali cure si possono fare e quali no al tempo del coronavirus - Il Governo stima 92mila contagi, il picco il 18 marzo
di Barbara Gobbi

EFFETTI COLLATERALI
25+ "Tutto è subito". Così il virus accelera gli ordini in alcune fabbriche dell'automotive
di Luca Orlando

ANALISI
25+ Perché le curve epidemiologiche devono preoccupare il resto d'Europa
di Biagio Simonetta

L'AUTOBIOGRAFIA
Van Basten, il campione fragile si mette a nudo
di Dario Ceccarelli

LE REGOLE SUGLI SPOSTAMENTI
Coronavirus, gli obblighi e i reati: le sanzioni solo dopo un processo - Assemblee condominiali soltanto in video

Pagina

HTML



Embedded objects

HTML: HyperText Markup Language

- **Linguaggio di markup** per la formattazione delle pagine (non linguaggio di programmazione!)
 - Usato per **disaccoppiare** la **struttura** di una pagina Web (definita dal markup) dalla sua rappresentazione puntuale (**stile**, definita dai fogli di stile CSS)
 - Fornisce **linee guida generali** per la rappresentazione del contenuto.
 - **Non specifica esattamente il formato e la posizione del testo**, lasciando ai CSS e alla interpretazione dei browser la definizione dei dettagli
- **Due browser potrebbero visualizzare lo stesso documento HTML in modo differente**
- Nonostante le proposte di modifiche ed altri standard (es. XML), a tutt'oggi HTML rimane il **“linguaggio del Web”**
 - La pagina HTML è un file di **solo testo ASCII** - testo **free-format**

Statement HTML



- Gli statement HTML sono racchiusi tra parentesi angolari, nella forma **<tag>**, e vengono terminate solitamente da un tag di chiusura nella forma **</tag>**
- **Schema di una pagina (documento) HTML**

```
<!DOCTYPE html>

<HTML>

  <HEAD>

    <TITLE>
      Titolo del documento
    </TITLE>

  </HEAD>

  <BODY>
    Corpo del documento
  </BODY>
</HTML>
```

Tag nel body



I tag presenti nel body servono per il controllo di:

- **intestazioni** (titoli di capitoli, di paragrafi eccetera)
 - Es. `<h1>Titolo principale</h1>` ←
- **strutture** di testo (testo indentato, paragrafi, eccetera)
- **aspetto** del testo (grassetto, corsivo, eccetera)
- **elenchi e liste** (numerate, generiche, di definizione)
- **tabelle**
- **moduli di form** (campi compilabili dall'utente, campi selezionabili, menu a tendina, pulsanti eccetera)
- **collegamenti ipertestuali** (**tag ancora** `<a> ... </a>`)
- inserimento di **immagini** o di **contenuti multimediali** (audio, video, animazioni eccetera)
- inserimento di **contenuti interattivi** (script, applicazioni esterne)

Definisce che è un titolo di livello 1, ma non come formattarlo

Oggetti 'embedded'

I contenuti **embedded** sono quelli che, per loro natura intrinseca, vengono embeddati, ossia incorporati, all'interno del contenuto

- Tipico oggetto embedded è l'immagine
- Uso di espliciti tag per le immagini embedded:

- Attributi

Es. Possibilità di allineare le immagini

Riferimento al file
pic_trulli.jpg
"src" unico attributo obbligatorio

Proviamo...



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

<https://www.w3schools.com/html/>

Easy Learning with "Try it Yourself" approach

Provate

HTML Introduction

HTML Headings

HTML Lists

HTML Tables

HTML Iframes

HTML CSS

Il protocollo HTTP

HyperText Transmission Protocol (HTTP)

- Protocollo applicativo di tipo request/reply basato sulla suite di protocolli TCP/IP
- Tutti i client e server Web (chiamati **client HTTP** e **server HTTP**) supportano il protocollo HTTP per poter scambiare richieste e risposte
- Usa **TCP (connection-oriented) come protocollo di trasporto**
 - Il client HTTP inizia la connessione TCP verso il server sulla porta 80
 - Il server HTTP accetta la connessione TCP dal client
 - Una volta stabilita la connessione, client e server HTTP accedono alla connessione (TCP) attraverso le loro socket (interfacce)
 - Messaggi HTTP scambiati tra client e server HTTP
 - Chiusura della connessione TCP
- *TCP offre un servizio di **trasferimento affidabile**: i messaggi di richiesta/risposta sono **consegnati integri** al destinatario → HTTP non si deve preoccupare di messaggi smarriti o non corretti (compito di TCP e livelli inferiori)*

HTTP è un protocollo stateless (senza stato)

- Il server HTTP non conserva nessuna informazione riguardante le richieste dei client passati
- *I protocolli che conservano lo stato sono complessi!*
 - La storia passata (lo stato) deve essere memorizzata
 - Se il server/client subiscono un crash, la vista dello stato può essere inconsistente e deve essere ristabilita

Connessioni HTTP: persistenti o non persistenti?

- Il client solitamente invia al server una **serie** di richieste
- **Connessioni non persistenti (HTTP 1.0 – prima versione)**
 - Ogni richiesta (e relativa risposta) viaggia su una connessione TCP separata
 - Il download di diversi oggetti richiede **connessioni multiple**
- **Connessioni persistenti (default da HTTP 1.1 in poi)**
 - Più oggetti possono essere inviati su una **singola connessione TCP** tra client e server

NOTA: una pagina Web include più oggetti

Connessioni non persistenti



Ipotesi: utente richiede URL www.someSchool.edu/someDepartment/home.index

*L'HTML contiene riferimenti
a 10 immagini jpeg*

1a HTTP client: inizia connessione TCP con il processo HTTP server su www.someSchool.edu su porta 80

2. HTTP client: invia messaggio di richiesta HTTP (contenente URL) attraverso la socket. Il messaggio richiede l'oggetto [someDepartment/home.index](http://www.someSchool.edu/someDepartment/home.index)

1b L'HTTP server sull'host www.someSchool.edu, in attesa di richieste per connessioni TCP sulla porta 80, "accetta" la connessione e notifica il client

3. HTTP server: riceve il messaggio di richiesta, crea il messaggio di risposta contenente l'oggetto richiesto, e manda il messaggio attraverso la socket

5. HTTP client: riceve il messaggio di risposta HTTP contenente il file HTML, fa il parsing e trova riferimenti a 10 oggetti JPEG

4. HTTP server: chiude la connessione TCP

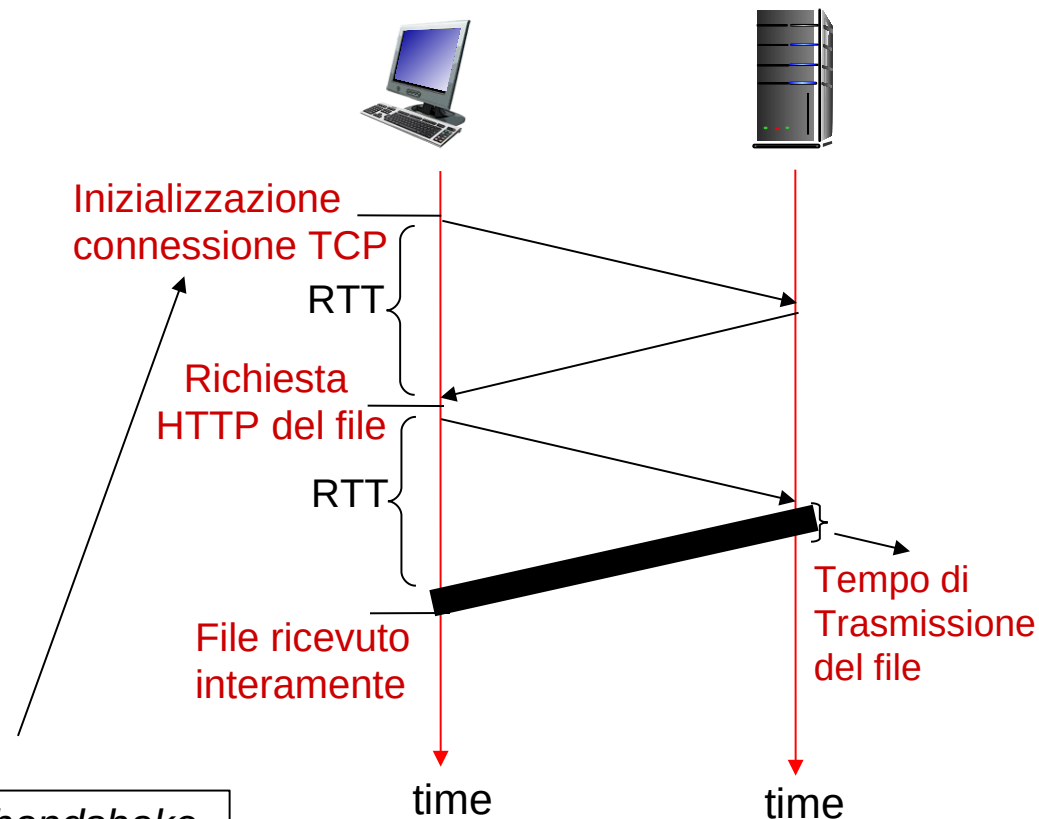
6. Gli step 1-5 sono ripetuti per ognuno degli oggetti JPEG

→ **11 connessioni TCP!**

HTTP non persistente: tempo di risposta

Stima tempo necessario tra la richiesta di un file HTML e la sua completa ricezione

- **RTT Round Trip Time (definizione):**
tempo impiegato da un singolo pacchetto per viaggiare dal client al server e ritorno
- **HTTP tempo di risposta:**
- 1 RTT per iniziare la connessione TCP
- 1 RTT tra la richiesta HTTP e l'arrivo dei primi byte di risposta
- Tempo di trasmissione del file
- Tempo di risposta per HTTP con connessioni non persistenti:
2RTT+ tempo trasmissione file



*Fase detta di Three-way handshake
(vedremo in TCP)*

HTTP Persistente



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

HTTP Non Persistente

- Richiede **2 RTT per ogni oggetto** da scaricare (trascuriamo il tempo di trasmissione)
- **Overhead** sul sistema operativo per ogni connessione TCP aperta
- **Onere per i server** che tipicamente servono richieste provenienti da centinaia di client diversi

HTTP Persistente

- Il server lascia la **connessione aperta** dopo aver inviato la risposta HTTP
- I successivi messaggi HTTP tra gli stessi client e server passano sulla **stessa connessione** già aperta
- Connessione chiusa dal server Web se rimane **inattiva per lasso di tempo** (configurabile)

Richiesta HTTP

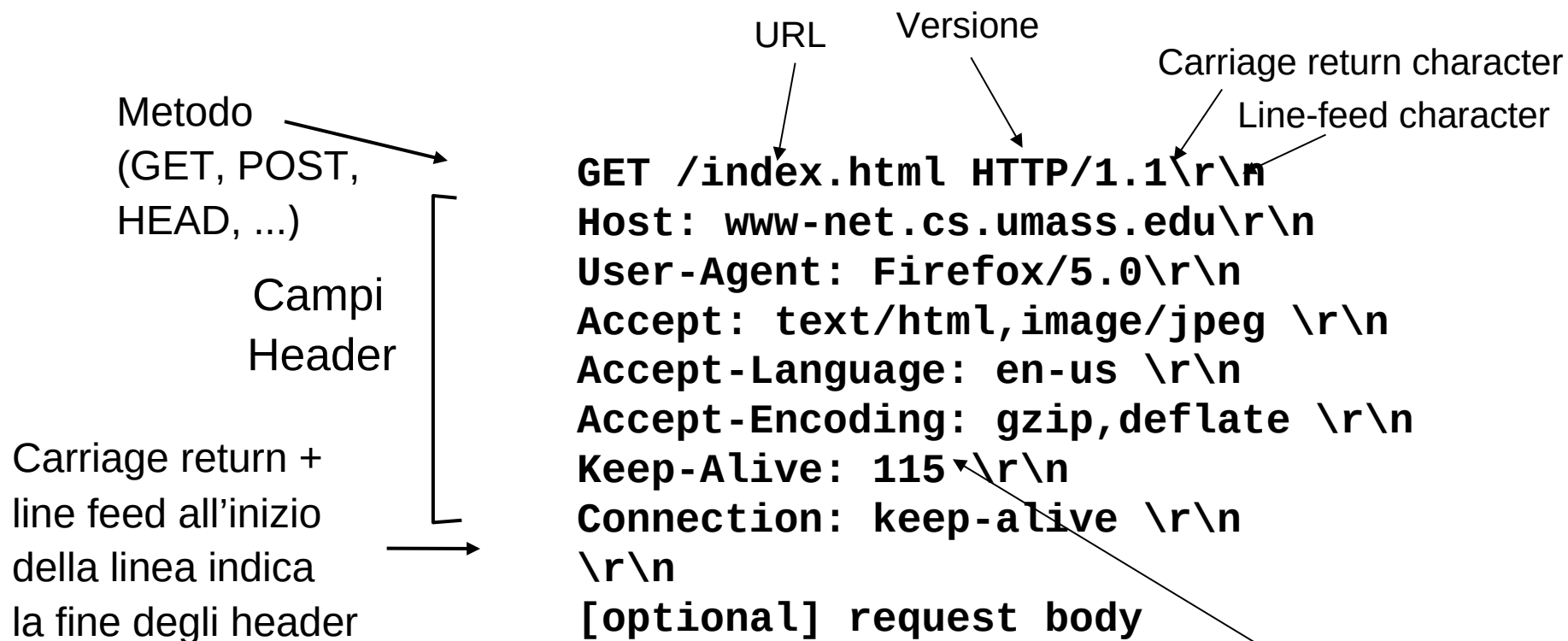


- **Due tipi di messaggi HTTP:**
 - *Richiesta HTTP*
 - *Risposta HTTP*
- Una **richiesta HTTP** comprende
 - *Metodo*
 - *URL*
 - *Identificativo della versione del protocollo HTTP (es. HTTP/1.1)*
 - *Insieme di extension header*
- Il **metodo** specifica il tipo di operazione che il client richiede al server: il metodo più comune è GET, per acquisire pagine Web
- Gli **header** contengono **informazioni aggiuntive**, quali la data e l'ora della comunicazione, il tipo di software utilizzato dal client, i tipi di dato che il browser è in grado di visualizzare, per un totale di circa 80-100 **header differenti**
- Il **messaggio di richiesta HTTP** è in **ASCII** (human-readable format)

Richiesta HTTP



Messaggio di richiesta HTTP ASCII



La connessione persistente viene chiusa dopo 115 secondi di inattività

Richiesta HTTP



Metodo	Richiesta	Versione
GET	Ricevere una risorsa dal server	≥ 1.0
HEAD	Ricevere il solo header di una risorsa	≥ 1.0
POST	Inviare dati al server	≥ 1.0
PUT	Inviare un oggetto al server	≥ 1.1
DELETE	Cancellare un oggetto dal server	≥ 1.1
TRACE	Individuare la catena dei server proxy	≥ 1.1

- **GET**: richiesta di un oggetto
- **POST**: la richiesta del client contiene dati nel campo body della richiesta (spesso le pagine Web includono input form: azione di submit → richiesta POST)
- **HEAD**: il client richiede che il server invii soltanto l'header della risposta senza l'oggetto (usato per debugging nei Web server)

Campi Header



- **User-agent**: browser utilizzato dall'utente
- **Accept**: tipo di oggetti che il client accetta
- **Accept-language**: preferenza della lingua
- **Accept-encoding, Accept-charset**
- **Host**: specifica host che ha la risorsa (HTTP 1.1)
- **Connection**: tipo di connessione richiesta dal client (persistente, non-persistente)
- **Keep-alive**: secondi di inattività prima della chiusura della connessione
- Altri header per garantire la **consistenza delle informazioni** (es., If-Match o If-Modified-Since)

Risposta HTTP



- Comprende, oltre al contenuto della risorsa richiesta, un **header** contenente l'identificativo della versione del protocollo HTTP, lo **status code**, l'**informazione di stato in forma testuale**, e altre informazioni di risposta
- Se la pagina richiesta oltre all'HTML contiene altri oggetti, ciascuno di essi sarà identificato da un URL per cui il browser dovrà inviare un esplicito messaggio di richiesta → **risposta HTTP per ogni oggetto embedded**

Status line (Status
Code + Phrase)

Campi
header

Body della risposta
con i dati, es., file
HTML richiesto

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

Risposta HTTP



- **Version:** versione del protocollo HTTP
- **Status code, phrase:** esito della richiesta (codice e frase)
- *Possibili codici (alcuni esempi)*
 - **2xx:** Successo
 - Es. **200 OK** – Richiesta servita con successo
 - **3xx:** Redirezione
 - Es. **301 Moved Permanently** – L'oggetto richiesto è stato spostato in modo permanente, la nuova locazione è specificata in questo messaggio (nell'header **Location:**)
 - **4xx:** Errore del client
 - Es. **400 Bad request** – messaggio di richiesta non compreso da questo server
 - Es. **404 Not Found** – Oggetto richiesto non trovato su questo server
 - **5xx:** Errore del server
 - Es. **504 HTTP Gateway Timeout**
 - Es. **505 HTTP Version Not Supported**

- **Headers**
 - **Connection**: tipo di connessione usata dal server
 - **Date**: data e ora della richiesta
 - **Server**: tipo di Web server e di sistema operativo
 - **Last-Modified**: data e ora creazione o modifica dell'oggetto (caching)
 - **Content-Length**: dimensione in byte dell'oggetto
 - **Content-Type**: tipo di oggetto (es. HTML, GIF, ...)
- **Body**: oggetto richiesto

Tipo MIME o Media Type



- HTTP trasferisce byte. Ma come fa il browser a sapere se quei byte sono HTML, JSON, un'immagine o un PDF?
- **MIME (Multipurpose Internet Mail Extensions)**
- Originariamente sviluppati per l'email, i tipi MIME sono oggi utilizzati in molti protocolli, incluso HTTP, per indicare la natura e il formato dei dati inviati
- Oggi più comunemente detto **Media Type**
- MIME definisce un sistema standard per descrivere il tipo e il formato del contenuto trasmesso, indicando come interpretare il contenuto trasmesso
- In HTTP compare negli header HTTP della risposta del server :

Content-Type (fondamentale)

Tipo MIME o Media Type



- Identificativo diviso **in due parti** usato per specificare il tipo di contenuto di una risorsa trasmessa
- Esempi di Content-Type:
 - Testo: text/html text/plain
 - Immagini: image/png, image/jpeg, image/gif
 - Audio, video: audio/mp3, video/mp4
 - File: application/pdf, application/zip
- Formato completo: type/subtype; parameter=value
 - Esempio Content-Type: text/html; charset=UTF-8
 - HTTP trasporta byte → Il browser deve sapere come interpretarli
 - charset=UTF-8 → come devono essere interpretati i byte come caratteri?
 - Il corpo della risposta contiene un documento HTML codificato come testo, e i byte devono essere interpretati usando la codifica UTF-8
 - Standard dominante sul web

Espresso come
type/subtype

Importanza:

- Compatibilità e interpretazione
- Sicurezza contro attacchi MIME sniffing

Interazione utente-server: cookie



Il server invia al client un "cookie" nella risposta **Set-cookie: session_id=1678**

Il client presenta il cookie nelle richieste successive: **cookie: session_id=1678**

Il server può confrontare il cookie presentato con i cookie da lui memorizzati per

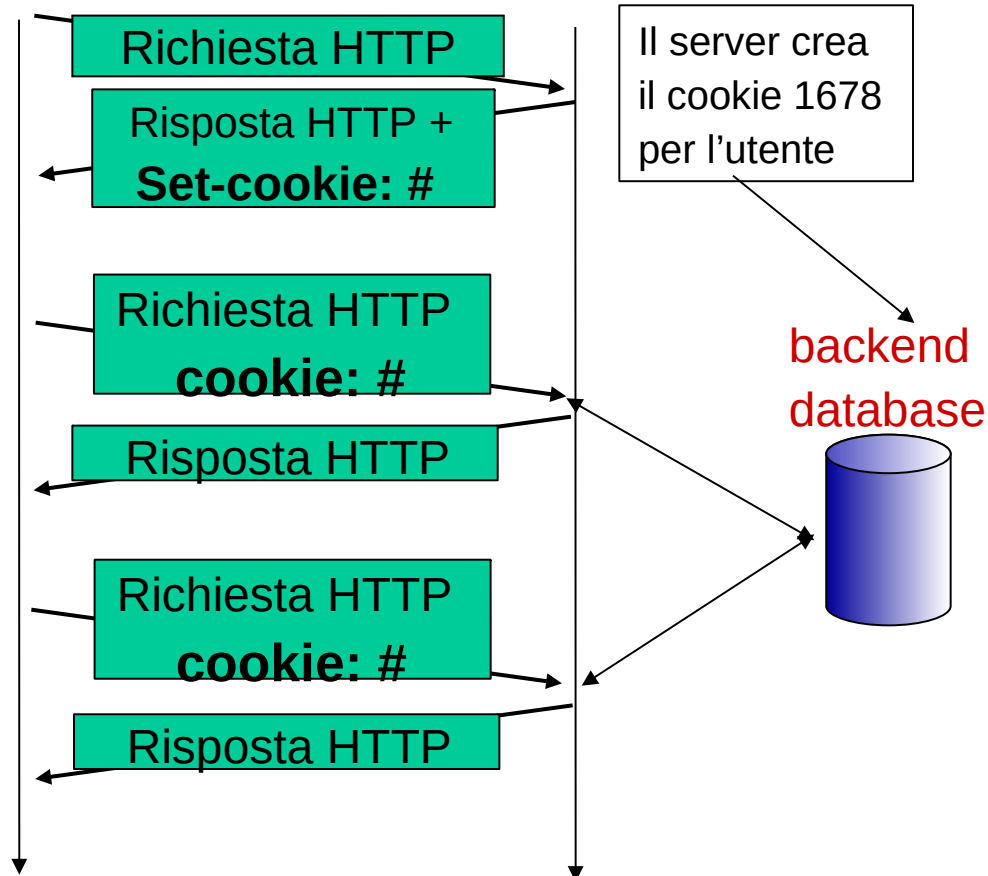
- **identificare** il client
- **ricostruire** le “**sessioni**” per seguire le preferenze e la navigazione degli utenti

Il browser inserisce il cookie in un “file dei cookie” nel file system locale, associato al sito visitato: il cookie verrà reinvio ad ogni successivo accesso allo stesso sito (anche dopo una settimana...)

L'utente può cancellare i cookie (funzione dei browser)

Client

Server



Cookie: un sistema di tracciamento



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Usato per **mantenere la sessione utente sopra HTTP** che è stateless
- Usati nei **meccanismi di autenticazione e gestione delle sessioni con il login**:
 - Consente all'utente di rimanere connesso durante la navigazione tra le pagine senza dover inserire nuovamente le credenziali
 - Consente all'utente di evitare il login al primo accesso
- Usato per la gestione ed il recupero del carrello virtuale in e-commerce
- Meccanismo usato per **monitorare l'attività dell'utente, pur senza registrazioni ad un sito**
 - Non conosco il nome dell'utente ma tutte la attività fatte sul sito (pagine visitate in quale ordine e orario)
- Usato per effettuare **Recommendation** su futuri acquisti
- ***Fonte di controversie: violazione privacy, vendita di profili utente a terze parti***

Cookie di prima e terza parte



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Cookie di prima parte: sono i cookie inviati al browser direttamente dal sito che si sta visitando - gestiti direttamente dal proprietario e/o responsabile del sito e usati principalmente per garantire il funzionamento tecnico o tenere traccia di preferenze espresse in merito all'uso del sito stesso.

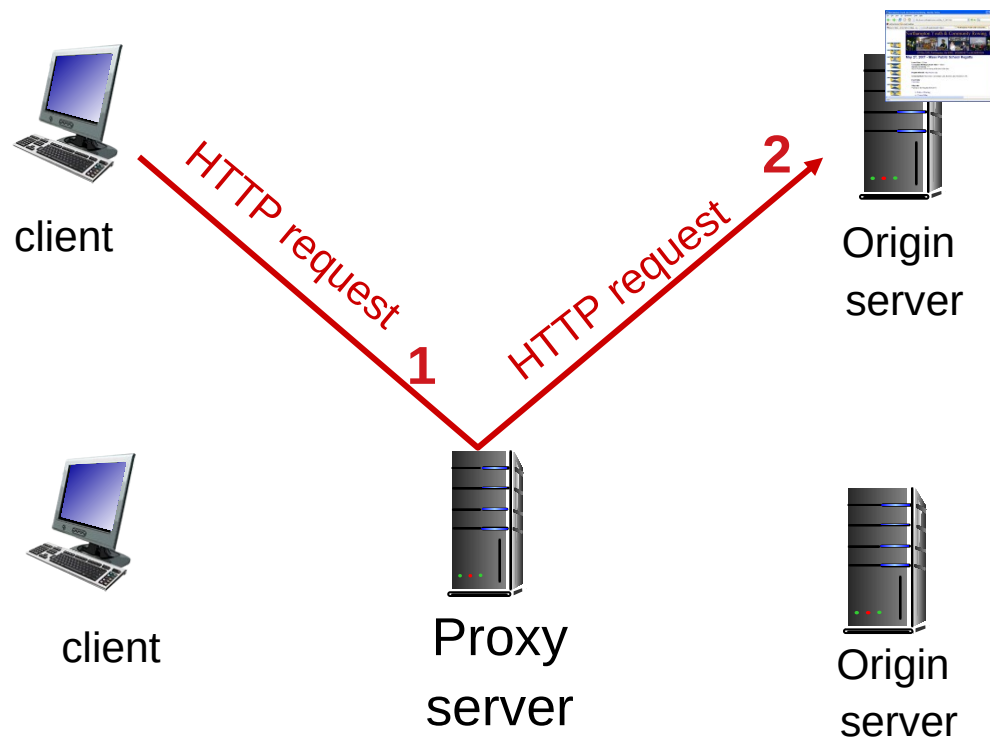
Cookie di terza parte: appartengono a domini diversi dal sito che si sta visitando. Questi cookie appaiono in genere quando le pagine web sono dotate di contenuti (come banner pubblicitari) da siti web esterni. Questo implica la possibilità di monitoraggio della cronologia di navigazione dell'utente, ed è spesso usato dagli inserzionisti per servire annunci rilevanti e personalizzati per ciascun utente.

- Es. supponiamo che un utente visiti www.example.org che contiene un annuncio da ad.foxytracking.com → cookie che appartiene al dominio della pubblicità (ad.foxytracking.com). Quando l'utente visita un altro sito Web, www.foo.com, che contiene un annuncio da ad.foxytracking.com/ → altro cookie appartenente a quel dominio (ad.foxytracking.com). Alla fine, entrambi questi cookie saranno inviati all'inserzionista quando si scaricano le loro pubblicità o visitando il loro sito Web → l'inserzionista può costruire una cronologia di navigazione degli utenti in tutti i siti che hanno i propri annunci

La maggior parte dei moderni browser Web contengono delle impostazioni di privacy che sono in grado di **bloccare i cookie di terze parti**.

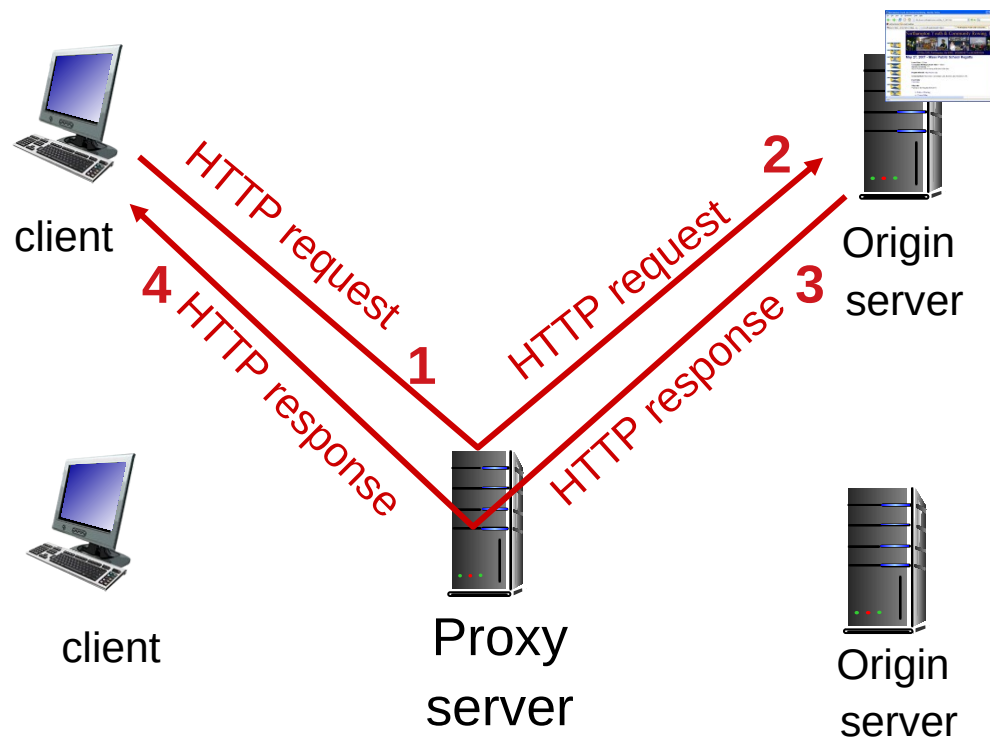
Web Caching e Proxy Servers

- **Obiettivo:** soddisfare le richieste HTTP dei client senza coinvolgere il Web server di origine (evitare overload del server, diminuire il tempo di risposta, ..)
- **Web Browser:** settato per **inviare** tutte le **richieste Web alla cache** (proxy server)
- **Oggetti già in cache:** vengono inviati al client direttamente dal proxy server
- **Oggetti non in cache:** richiesti al server di origine, **memorizzati in cache** e poi inviati al client



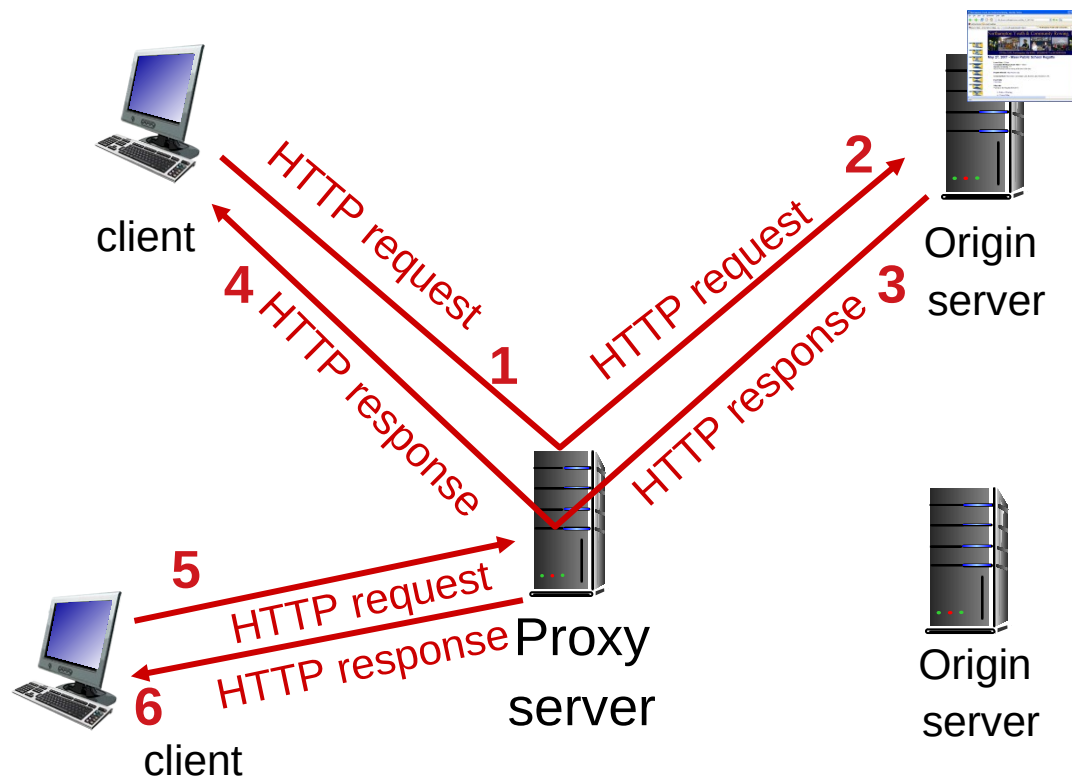
Web Caching e Proxy Servers

- **Obiettivo:** soddisfare le richieste HTTP dei client senza coinvolgere il Web server di origine (evitare overload del server, diminuire il tempo di risposta, ..)
- **Web Browser:** settato per **inviare** tutte le **richieste Web alla cache** (proxy server)
- **Oggetti già in cache:** vengono inviati al client direttamente dal proxy server
- **Oggetti non in cache:** richiesti al server di origine, **memorizzati in cache** e poi inviati al client



Web Caching e Proxy Servers

- **Obiettivo:** soddisfare le richieste HTTP dei client senza coinvolgere il Web server di origine (evitare overload del server, diminuire il tempo di risposta, ..)
- **Web Browser:** settato per **inviare** tutte le **richieste Web alla cache** (proxy server)
- **Oggetti già in cache:** vengono inviati al client direttamente dal proxy server
- **Oggetti non in cache:** richiesti al server di origine, **memorizzati in cache** e poi inviati al client
- **Proxy hit rate:** percentuale di richieste soddisfatte dal proxy



Altro sul Web Caching



- Proxy server: agisce **sia da client che da server**
 - server per il client che fa richieste
 - client nei confronti del server di origine
- **Tipicamente acquistato e installato da un ISP**
 - Es. Università, aziende, ISP residenziali
- **Motivi all'origine:**
 - **Ridurre tempi risposta** per le richieste dei client
 - Internet è piena di cache: rende possibile ai content provider meno potenti di **distribuire più efficacemente le risorse**
 - **Diminuire il traffico globale** sulla rete → migliori prestazioni
 - **Ridurre il traffico sui link di accesso** a internet

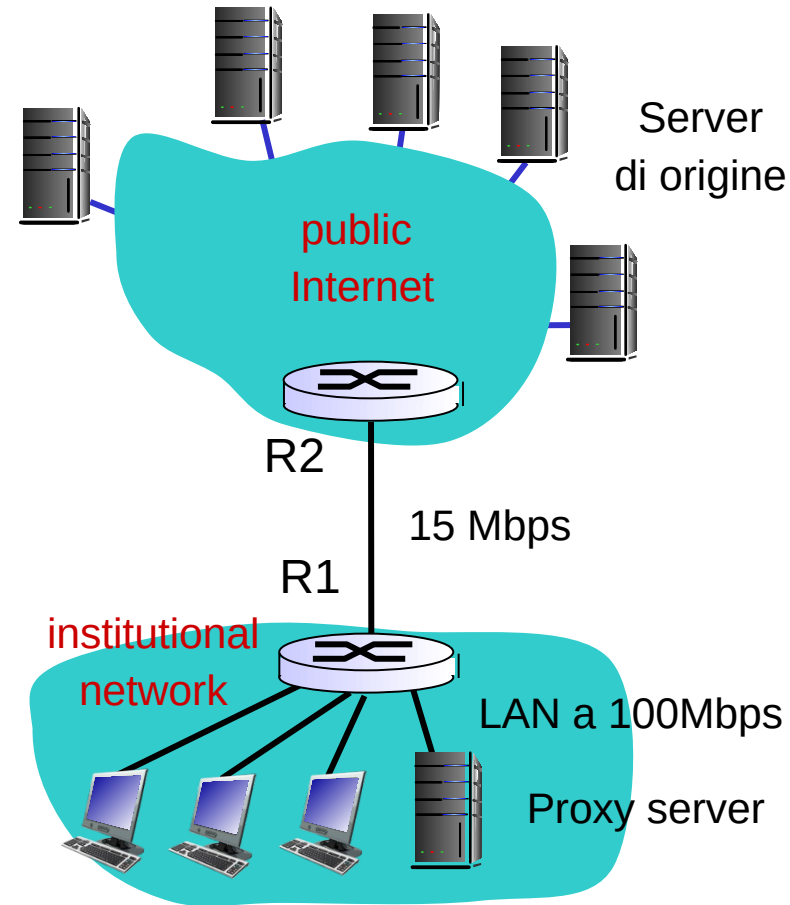
Vediamo con un esempio

Esempio

Ipotesi

- Dimensione oggetti media: 1Mbit
- Request rate medio dai browser ai server di origine: 15 req/sec
- Velocità link R1 - R2: 15 Mbps
- Velocità LAN: 100 Mbps

Qual è l'intensità di traffico (utilizzo) sulla rete LAN e sul link R1-R2?



Esempio

Intensità di traffico – In generale se:

- a = frequenza media di pacchetti scambiati (request rate medio)
- L = dimensione in bit dei pacchetti
- R = velocità trasmissione link (bit/s)

→ La/R = intensità traffico

Se supera 1, la rete si congestiona

Torniamo al nostro esempio

Intensità di traffico (senza proxy)

$$(15 \text{ req/s}) * (1 \text{ Mbit}) / (100 \text{ Mbps}) = 0,15$$

LAN: ok

$$(15 \text{ req/s}) * (1 \text{ Mbit}) / (15 \text{ Mbps}) = 1$$

Link R1-R2: rischio congestione

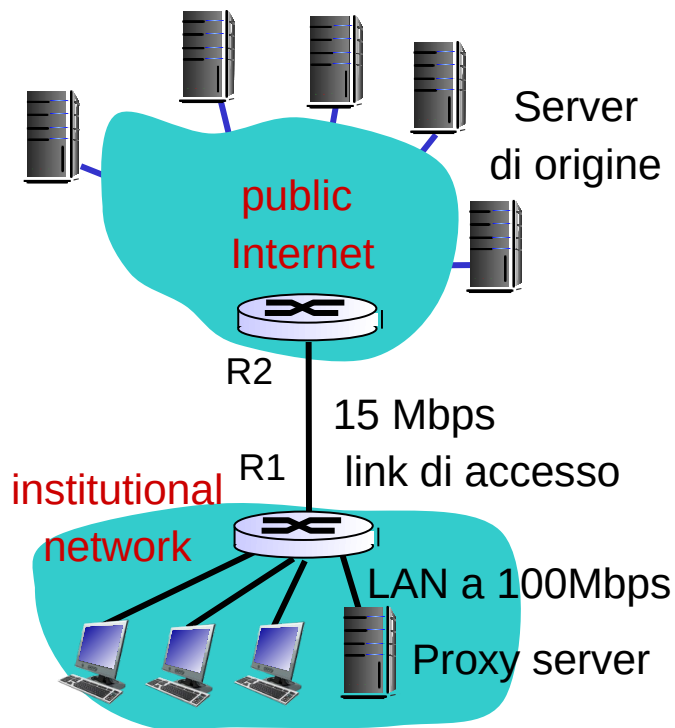
- **Soluzione:** proxy server (ipotesi hit rate del 30% - soddisfatte dal proxy)

- Solo 70% richieste passa sul link di accesso

$$(15 \text{ req/s}) * 0,7 * (1 \text{ Mbit}) / (15 \text{ Mbps}) = 0,7$$

Link R1-R2: OK

**Soluzione
Economica**



GET Condizionale

Potenziale problema: se la copia in cache **non è aggiornata** (*stale*) rispetto all'originale sul server ?

Meccanismo del **GET condizionale**

Obiettivo: non inviare l'oggetto se la cache ha una versione **aggiornata** (*up-to-date*)

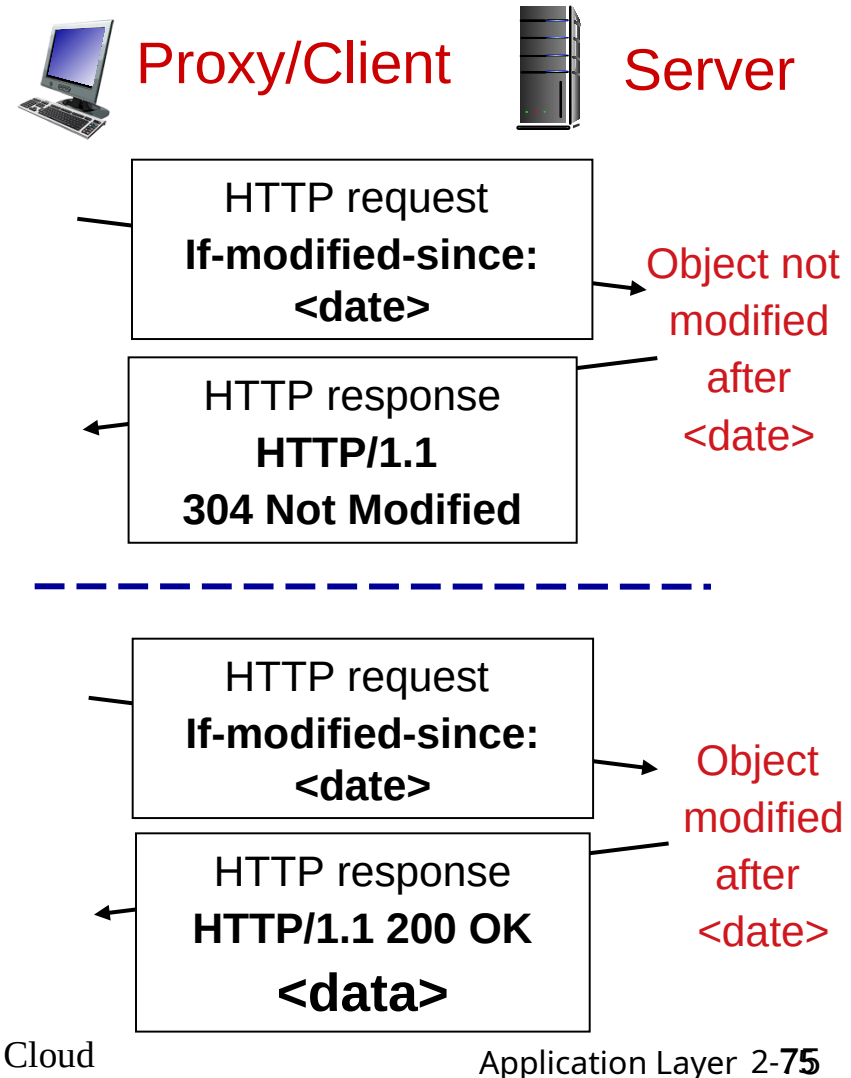
- No ritardo di trasmissione oggetto
- Minore utilizzo dei link

Proxy: invia la data della copia in cache nella richiesta HTTP

If-modified-since: <date>

Server: la risposta HTTP non contiene l'oggetto se la copia è aggiornata:

HTTP/1.1 304 Not Modified

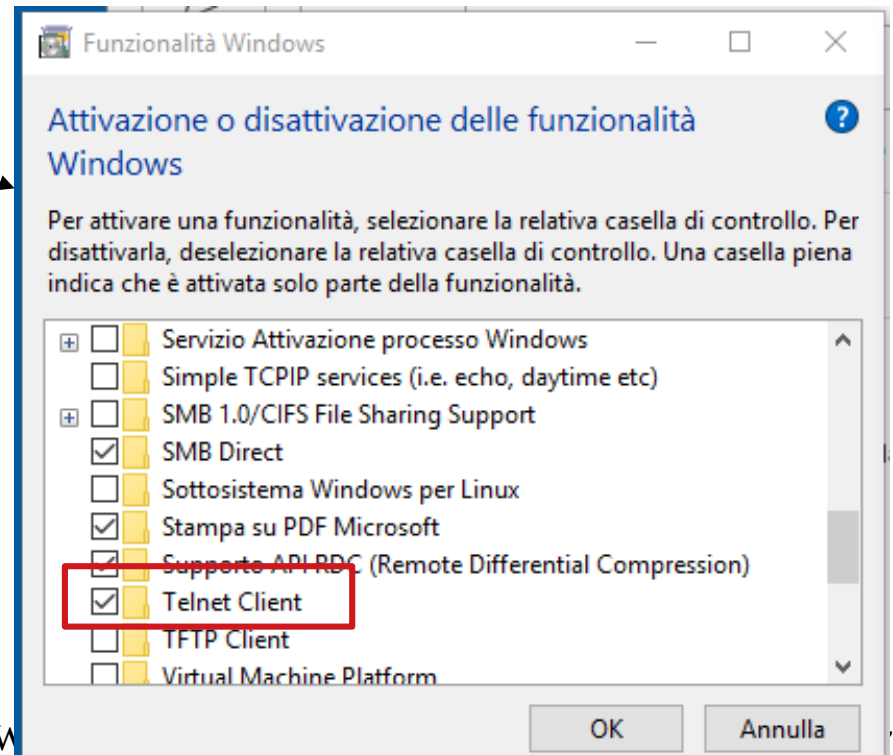


Uso Telnet

- **Telnet:** **protocollo applicativo** utilizzato per implementare un metodo standard di interfacciamento tra un dispositivo terminale (utente alla tastiera) e dei processi interattivi eseguiti in una macchina remota (**collegamento terminale-processo**)
 - Preinstallato sui più comuni SO - **da abilitare su Windows**

Aprire Funzionalità Windows
(Cercatelo da Start)

```
Ctrl Prompt dei comandi - telnet
Microsoft Telnet Client
Il carattere di Escape è 'CTRL++'
Microsoft Telnet> _
```



Esempio uso Telnet HTTP client-side



1. Aprire un terminale e collegarsi via Telnet ad un Web server (processo):

```
telnet gaia.cs.umass.edu 80
```

Aprire una TCP connection alla porta 80 (default HTTP server port) dell'host gaia.cs.umass.edu

Qualunque cosa successiva sarà mandata alla porta 80 di gaia.cs.umass.edu

2. Scrivere una richiesta HTTP GET:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
```

```
Host: gaia.cs.umass.edu
```

Con questo (fare invio due volte), Verrà inviata questa richiesta HTTP GET minimale (ma completa) al server Web su gaia.cs.umass.edu

Perchè?

3. Guarda il messaggio di risposta inviato dal Server Web!

Risposta ricevuta



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

HTTP/1.1 200 OK

Date: Sun, 23 Mar 2025 10:12:55 GMT

Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3

X-Powered-By: PHP/7.4.33

Set-Cookie: DevMode=0

Transfer-Encoding: chunked

Content-Type: text/html; charset=UTF-8

X-xxx: non standard header

X-Powered-By: often included in responses
constructed via a particular scripting technology

Indica che il corpo del messaggio viene inviato in
blocchi (chunk) anziché in un'unica parte continua

<!DOCTYPE HTML>

<html>

<head>

<title>Interactive Problems, Computer Networking: A Top Down Approach</title>

<meta charset="utf-8" />

...

<link href="custom.css" rel="stylesheet" type="text/css"/>

</head>

<body>

<script type="text/javascript" src="https://code.jquery.com/jquery-3.5.1.min.js"></script>

....

Dopo un pò di tempo appare la scritta
Connection closed by foreign host

HTTP/2.0



- Nasce per **migliorare le prestazioni** del Web
- Standardizzato nel **2015**
- **Obiettivi principali:**
 - ridurre latenza
 - ridurre overhead dei messaggi
 - permettere download paralleli efficienti
- **HTTP/2 non cambia il modello HTTP**
 - cambia il modo in cui i messaggi vengono trasmessi
- Principali cambiamenti in HTTP/2
 - Uso di **frame binari**
 - **Multiplexing** con stream contemporanei (vera innovazione)

1. Protocollo binario vs. testuale

HTTP/1.1: Utilizza un formato testuale, il che rende il parsing più semplice ma meno efficiente in termini di elaborazione e dimensione dei messaggi

HTTP/2: Utilizza frame binari: dati binari organizzati in piccole unità ('frame')

Riduce overhead e consente una gestione più efficiente dei dati durante il trasferimento

2. Compressione degli header

HTTP/1.1: Gli header vengono inviati in chiaro in ogni richiesta e risposta, aumentando l'overhead, specialmente con header ripetitivi.

HTTP/2: Utilizza un meccanismo di compressione degli header, riducendo significativamente il volume di dati inviati e migliorando le performance

3. Server Push

HTTP/1.1: Non dispone di un meccanismo nativo per inviare risorse non richieste.

HTTP/2: Consente al server di "spingere" (push) risorse aggiuntive al client prima che quest'ultimo le richieda, migliorando i tempi di caricamento

Es. il client chiede index.html e il server anticipa style.css e script.js

4. Multiplexing su singola connessione

HTTP/2 introduce il concetto di **stream**

Una stessa connessione TCP può trasportare **più stream contemporaneamente**

Confronto con HTTP/1.1: una connessione TCP gestisce le richieste in sequenza

Esempio: una pagina Web richiede HTML, CSS, JavaScript, immagine

Sequenza sulla connessione:

- Request HTML → Response HTML
- Request CSS → Response CSS
- Request JS → Response JS
- Request Image → Response Image

Se una risposta è lenta, blocca tutte le altre

Meccanismo chiamato **Head-of-line blocking**

HTTP/2.0

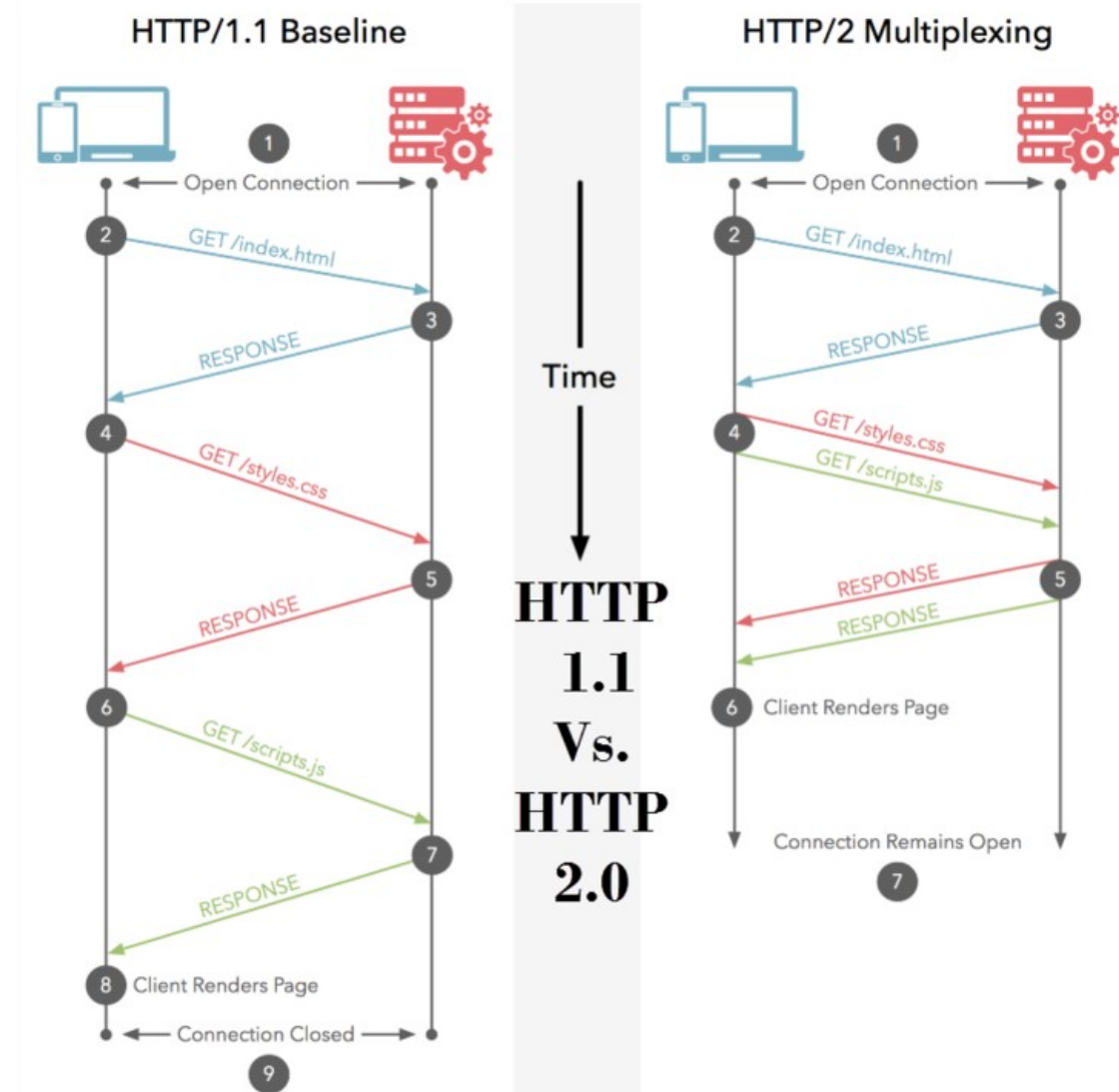
4. Multiplexing su singola connessione

HTTP/2 introduce il concetto di **stream**: uno stream è una sequenza logica di dati che appartiene a una richiesta HTTP

HTTP/2 divide i dati in **frame binari**: i frame vengono interallacciati e spediti sulla stessa connessione contemporaneamente

Ogni stream viene associato ad un identificatore univoco **stream ID**

Il client **ricostruisce** ogni risposta usando lo stream ID



Evoluzione più recente di HTTP

Standardizzato nel 2022
Supportato da molti browser
ma ancora in fase di adozione

- **Protocollo di Trasporto:**
 - HTTP/3 non utilizza più TCP ma **QUIC**, un protocollo di trasporto basato su **UDP** che integra funzionalità tipiche di **TCP** (come l'affidabilità della consegna e il controllo della congestione)
 - **QUIC (Quick UDP Internet Connections)** sviluppato originariamente da Google
- **Vantaggi di QUIC:**
 - **Multiplexing avanzato:** Consente di inviare più flussi di dati in parallelo su una singola connessione senza che la perdita di un pacchetto in un flusso blocchi gli altri flussi
 - **Riduzione della latenza:** Riduce il tempo necessario per stabilire una connessione sicura rispetto a TCP
 - **Sicurezza Integrata**
 - **TLS 1.3 Integrato:** QUIC incorpora TLS 1.3 direttamente nel protocollo, garantendo che tutte le comunicazioni siano crittografate sin dal primo scambio di dati

HTTP/3.0

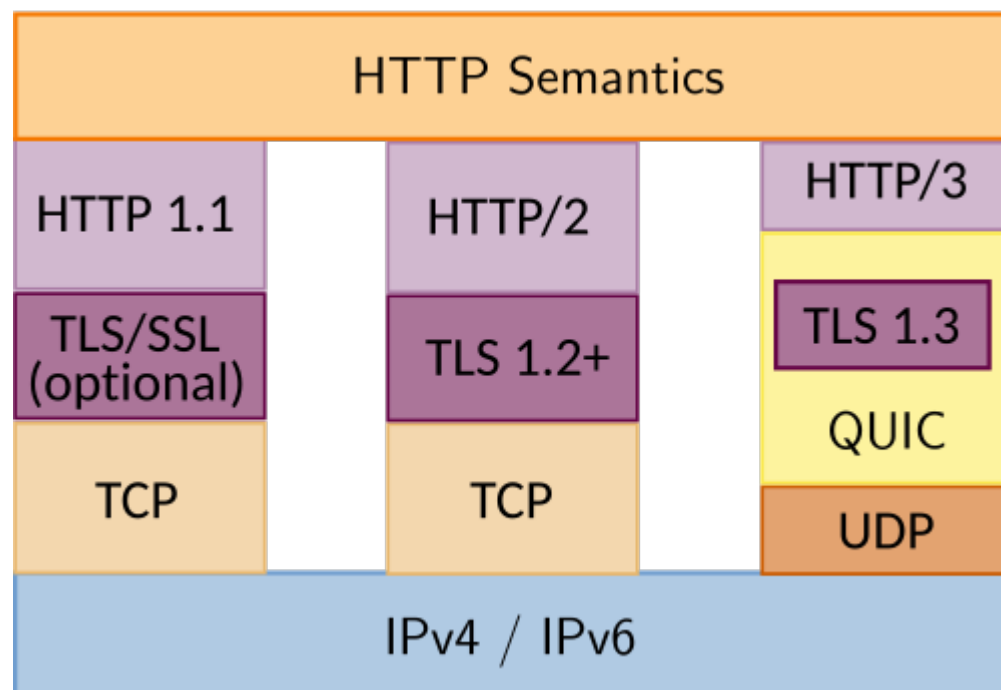
Mentre HTTP/2 si appoggiava a TCP e TLS per la sicurezza, HTTP/3 integra il trasporto e la cifratura direttamente nel protocollo QUIC (basato su UDP)

HTTP/2

Se un pacchetto viene perso, TCP interrompe la trasmissione di tutti gli altri dati in attesa della ritrasmissione di quel singolo pacchetto (ancora effetto Head-of-Line Blocking)

HTTP/3

QUIC/UDP: QUIC gestisce la perdita di pacchetti a livello di singola risorsa (stream). Es. se un'immagine si perde, il file CSS continua a essere scaricato senza ritardi



HTTPS HyperText Transfer Protocol Secure

- Versione sicura di HTTP, porta standard 443
- Utilizza crittografia TLS (Transport Layer Security) per proteggere le comunicazioni:
 - **Riservatezza** → I dati vengono crittografati, impedendo a terzi di intercettarli
 - **Integrità** → I dati non possono essere alterati durante il trasferimento
 - **Autenticazione** → L'utente è certo di comunicare col sito reale e non sito fraudolento

Funzionamento HTTPS

- 1) Il browser si collega al server tramite HTTPS (porta 443)
- 2) Inizia il TLS handshake: il browser invia un messaggio con le versioni TLS e gli algoritmi supportati
- 3) Il server risponde e invia il certificato digitale
- 4) Il browser verifica il certificato (CA, dominio, validità)
- 5) Client e server negoziano una chiave di sessione condivisa (Fine TLS handshake)
- 6) La connessione diventa cifrata
- 7) I dati HTTP vengono scambiati in modo sicuro

Certificati SSL/TLS



- **SSL (Secure Sockets Layer)** → Predecessore TLS, tecnologia ormai deprecata per garantire la sicurezza di una connessione a Internet mediante crittografia dei dati inviati tra un sito web e un browser (o tra due server)
- **TLS (Transport Layer Security)** → Versione aggiornata e più sicura di SSL. Si mantiene il termine SSL nel nome dei certificati (SSL/TLS)
- Per usare HTTPS, un server deve avere un **certificato SSL/TLS e installarlo**
 - Il certificato viene **emesso da un'Autorità di Certificazione (CA)** come DigiCert, GlobalSign, Sectigo, ...
 - Il certificato viene installato sul server
 - Modalità di installazione dipende dal server (Apache, Nginx)
 - Si abilita il redirect del traffico da HTTP a HTTPS

"E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."